

MAT1252

Mathematics for Computing

Lecture 04

Karnaugh Maps and Applications

Outline

- 01. Karnaugh maps
- 02. 3-variable Karnaugh maps
- 03. 4-variable Karnaugh maps
- 04. Minimal sum of products
- 05. SOPs, CSOPs and Min SOPs
- 06. Finding Min SOPs from K-maps
- 07. Design Problem Type 1: From truth table to circuit
- 08. Design Problem Type 2: Simplifying a given circuit
- 09. Application: 7-segment display
- 10. NAND gate
- 11. The universality of NANDs
- 12. Converting an AND-OR circuit to NANDs only

Lecture's Major Objectives

After completing this section, students should be able to:

- use a Karnaugh map to find a minimal sum of products
- given a sum of products, draw the corresponding Karnaugh map
- given a truth table, construct the corresponding Karnaugh map
- design a circuit specified by a truth table
- given a circuit, design a simpler equivalent circuit
- given a sum of products, find an equivalent expression using NANDs
- given an AND-OR circuit, draw an equivalent NAND circuit

Karnaugh maps

- A Karnaugh map is another way of **representing Boolean expressions**.
- A Karnaugh map (K-map) contains the same information as a **truth table**.
- Karnaugh maps are used to **simplify Boolean expressions** so that they can be implemented in circuits using the smallest number of logic gates.
- Karnaugh maps are **arranged as a grid**, which makes it easier to spot patterns in expressions.
- We will consider 3-variable and 4-variable Karnaugh maps.

Karnaugh maps

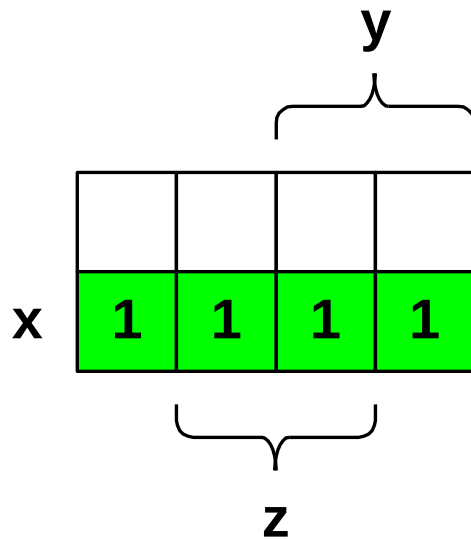
A typical 3-variable Karnaugh map is represented as:

		y'		y	
		0	0	1	1
x'	0	$x'y'z'$ 000	$x'y'z$ 001	$x'yz$ 011	$x'yz'$ 010
x	1	$xy'z'$ 100	$xy'z$ 101	xyz 111	xyz' 110
		0	1	1	0
		z'	z	z	z'

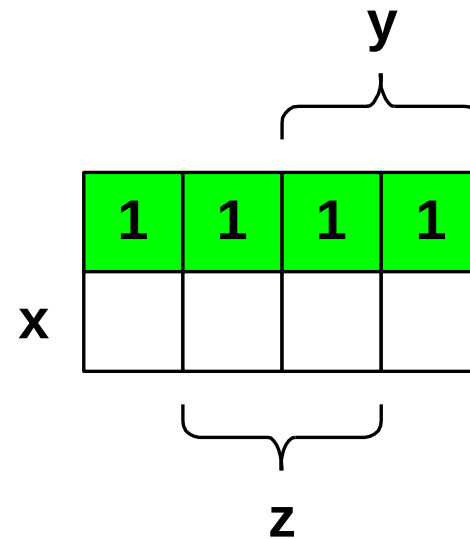
3-variable Karnaugh maps

A simple way to understand K-maps is to consider rows and columns and the products that they represent.

First we look at the rows. The bottom row of cells represents x and the top row represents x' , as shown in the figures.

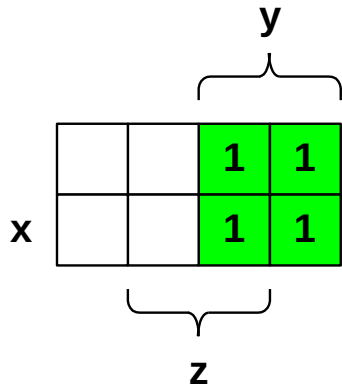


This row represents x

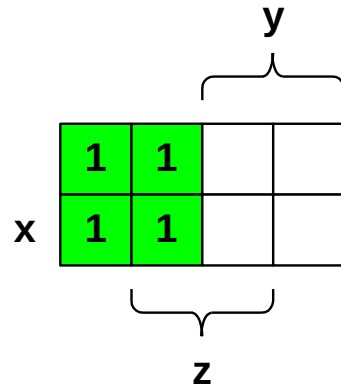


This row represents x'

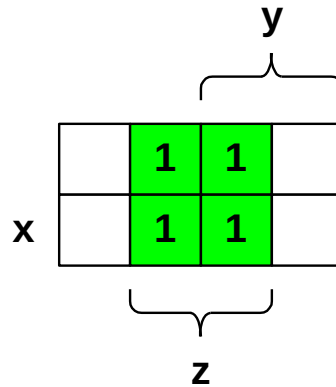
3-variable Karnaugh maps



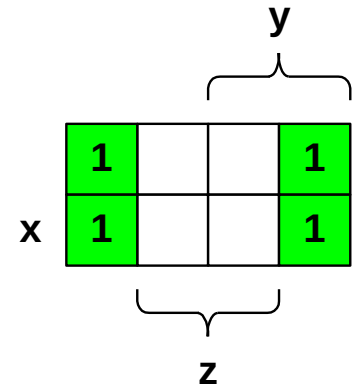
These columns
represents y



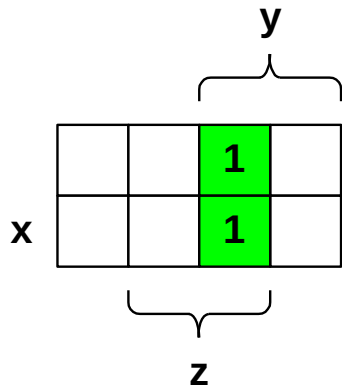
These columns
represents y'



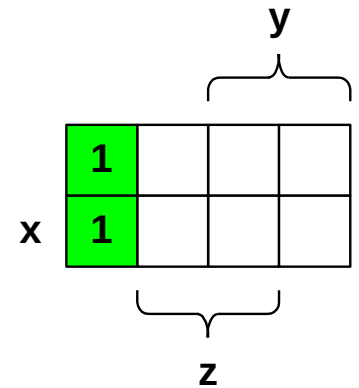
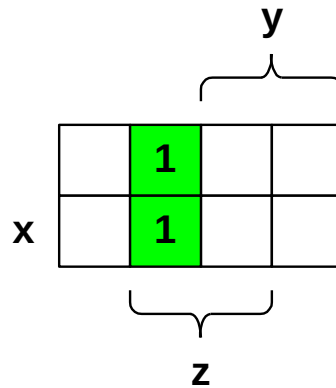
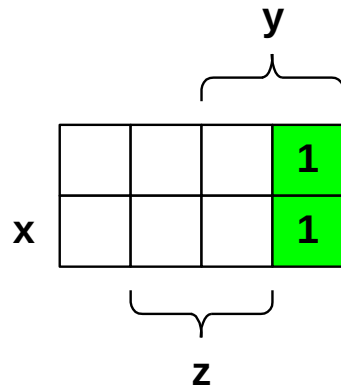
These columns
represents z



These columns
represents z'

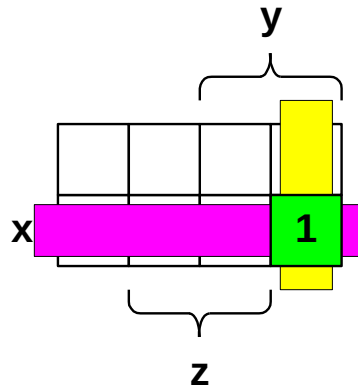


This column
represents yz

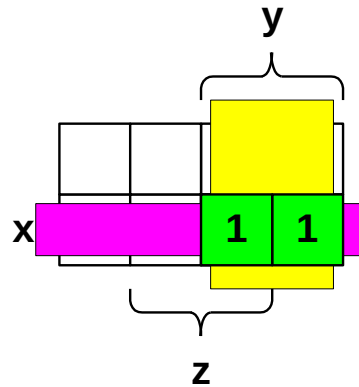


3-variable Karnaugh maps

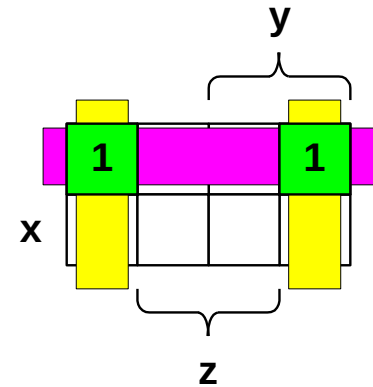
Example



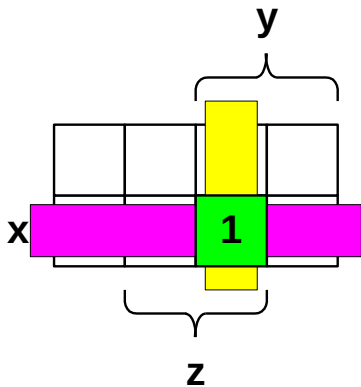
This single cell
represents xyz'



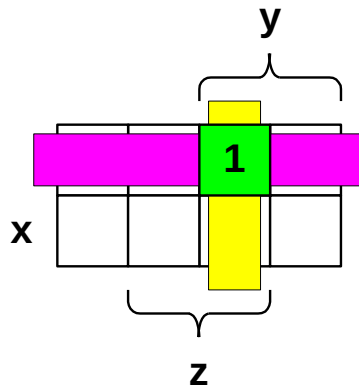
The intersection
represents xy



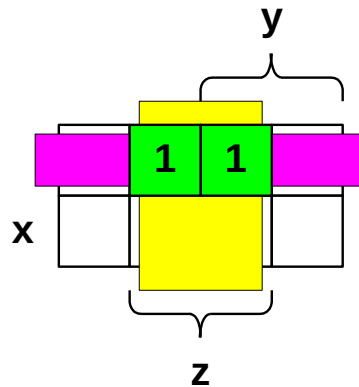
The intersection
represents $x'z'$



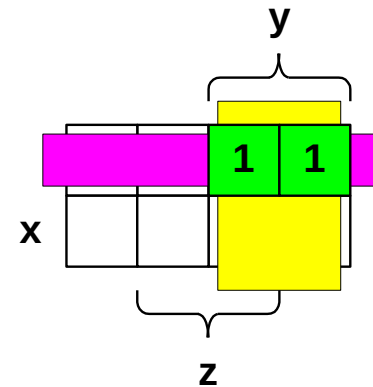
The intersection
represents xyz



The intersection
represents $x'yz$



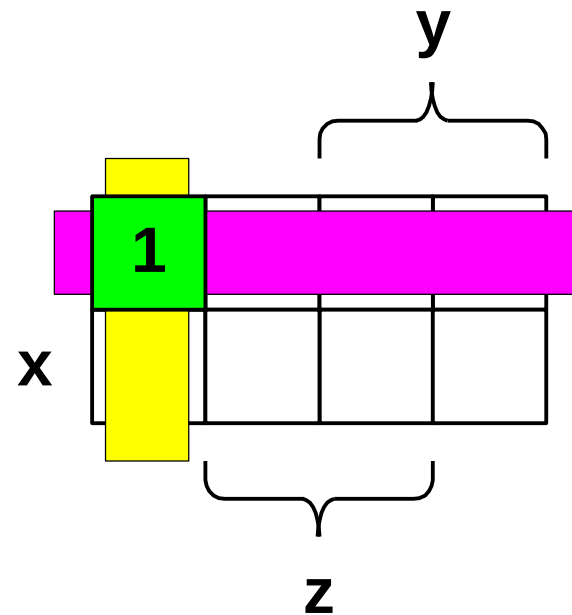
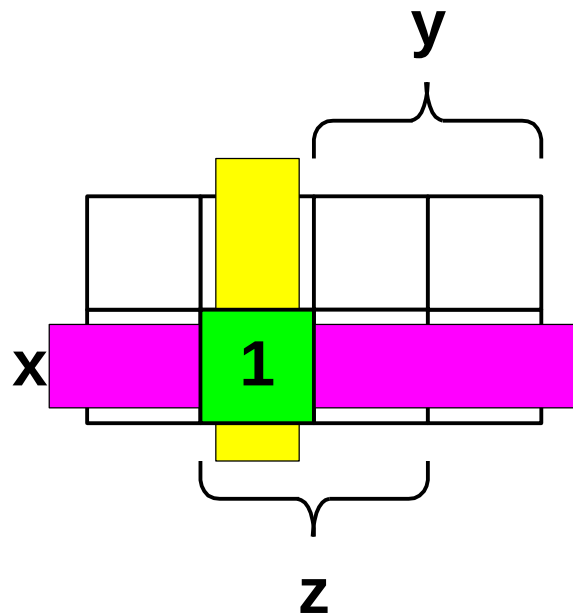
The intersection
represents $x'z$



The intersection
represents $x'y$

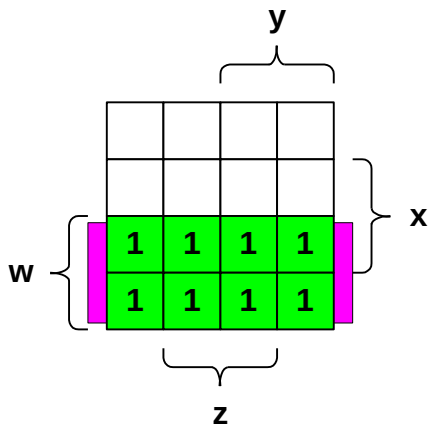
3-variable Karnaugh maps

Example

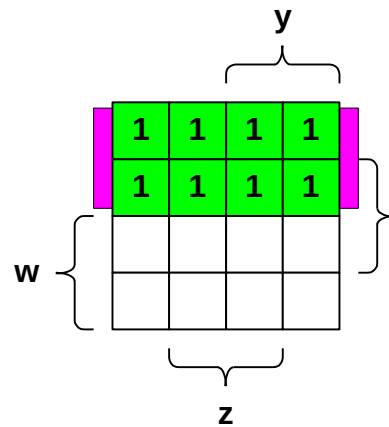


4-variable Karnaugh maps

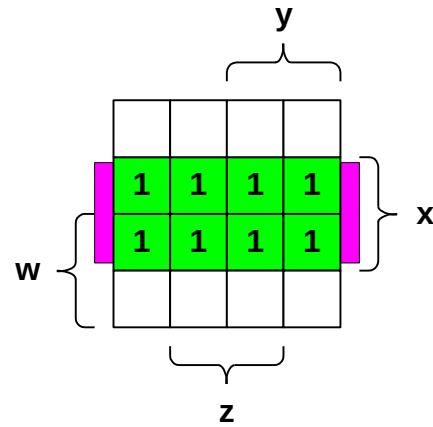
We will consider rows and columns:



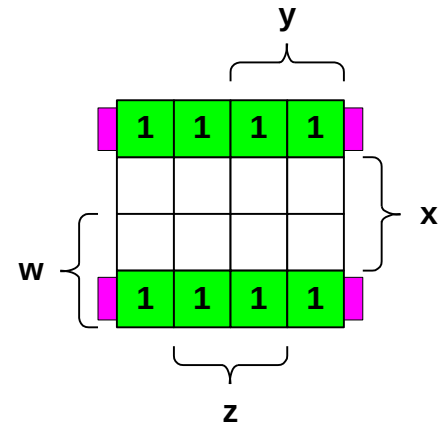
Those rows
represent w



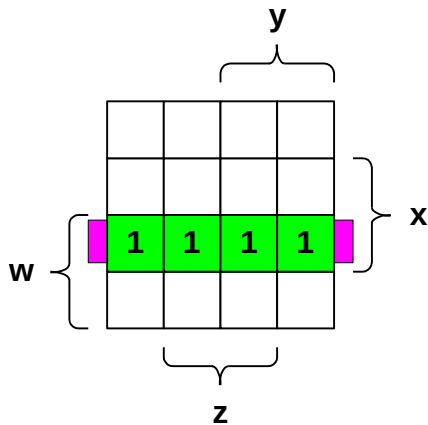
Those rows
represent w'



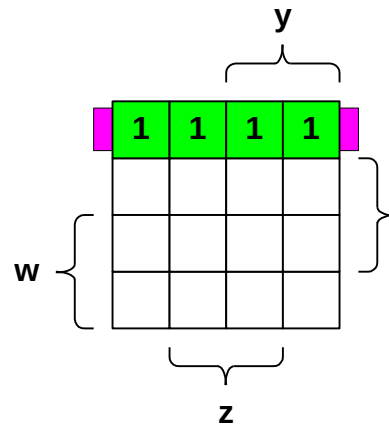
Those rows
represent x



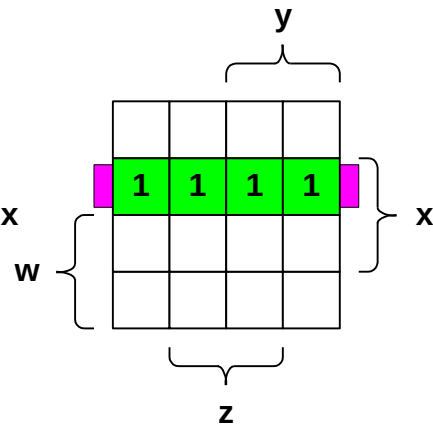
Those rows
represent x'



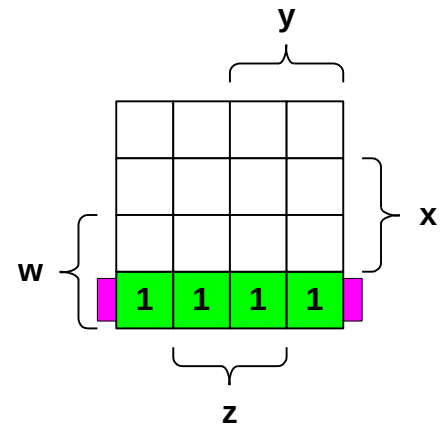
This row represent
 wx



This row represent
 $w'x'$

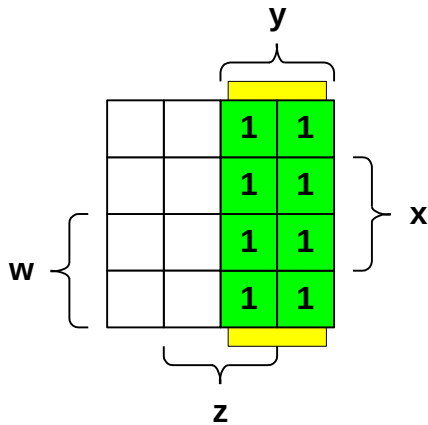


This row represent
 $w'x$

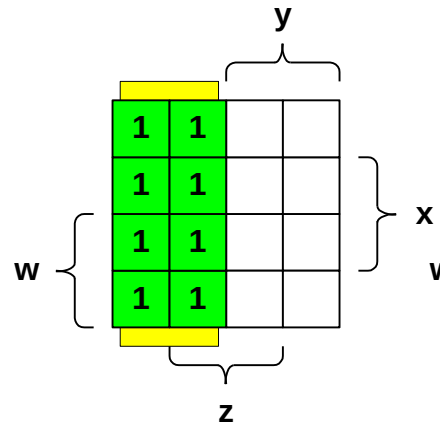


This row represent
 wx'

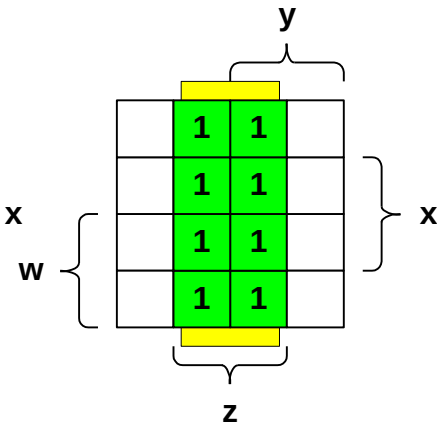
4-variable Karnaugh maps



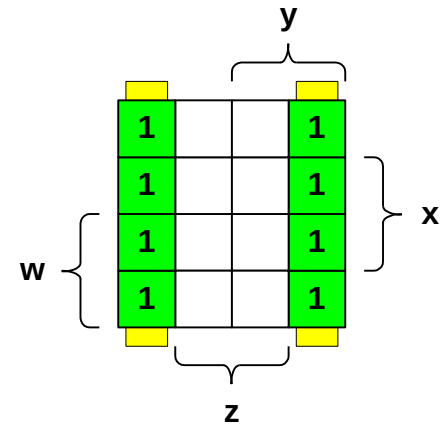
These columns represent y



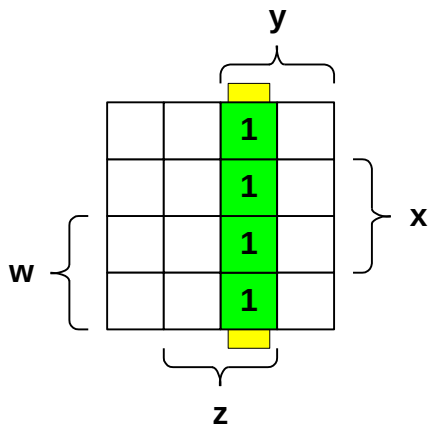
These columns represent y'



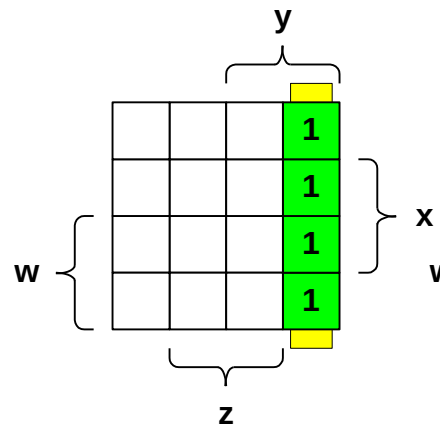
These columns represent z



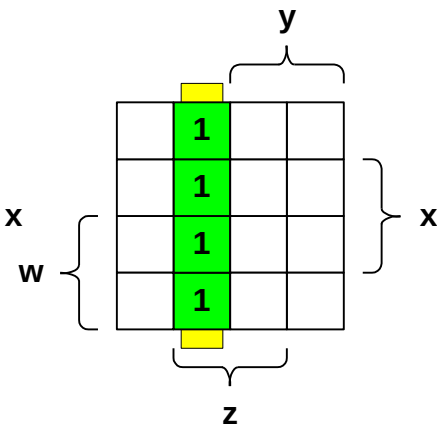
These columns represent z'



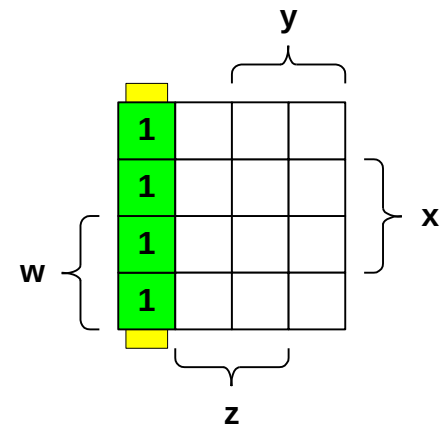
This column represents yz



This column represents yz'



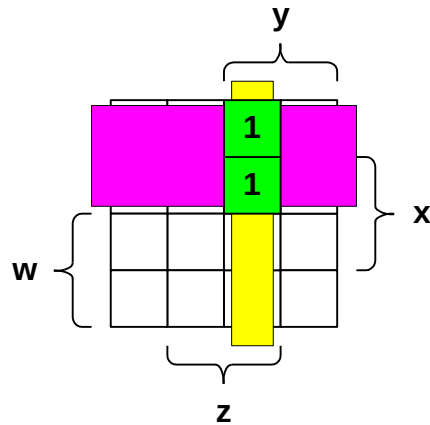
This column represents $y'z$



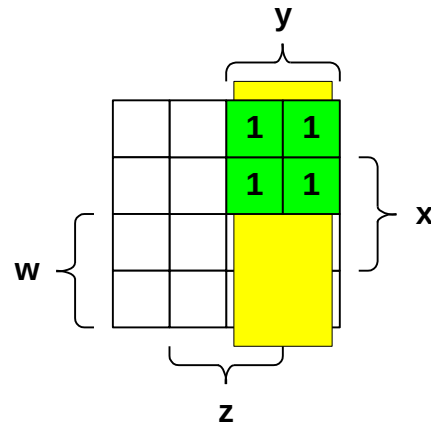
This column represents $y'z'$

4-variable Karnaugh maps

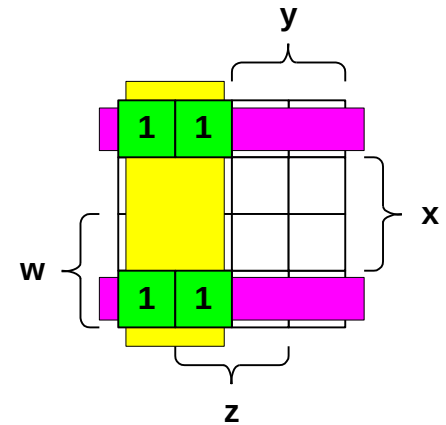
Example



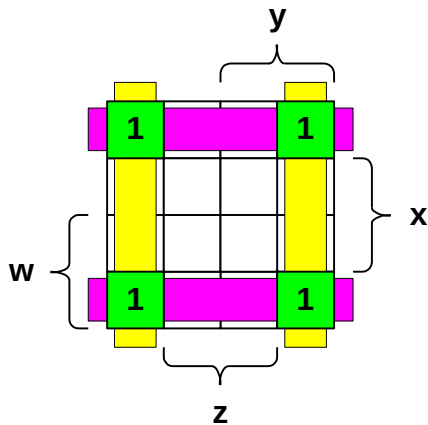
These 2 cells
represent $w'yz$



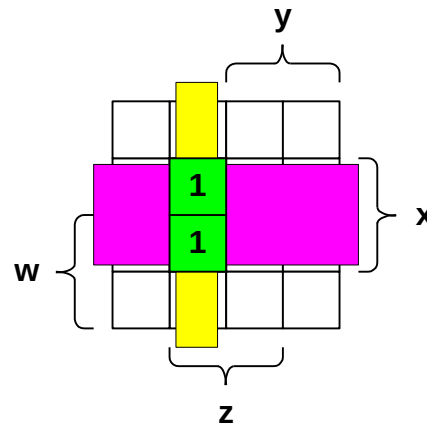
These 4 cells
represent $w'y$



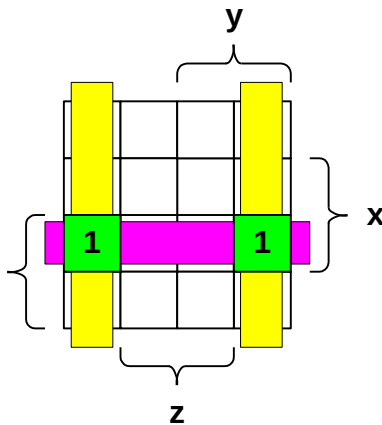
These 4 cells
represent $x'y'$



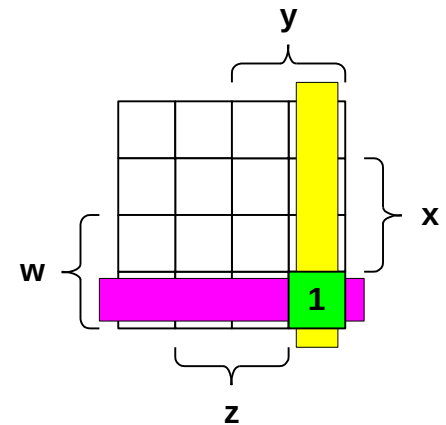
These 4 cells
represent $x'z'$



These 2 cells
represent $xy'z$



These 2 cells
represent wxz'



This cell represent
 $wx'yz'$

Aim of Karnaugh maps

We want to be able to take a Karnaugh map and represent it as a Boolean expression

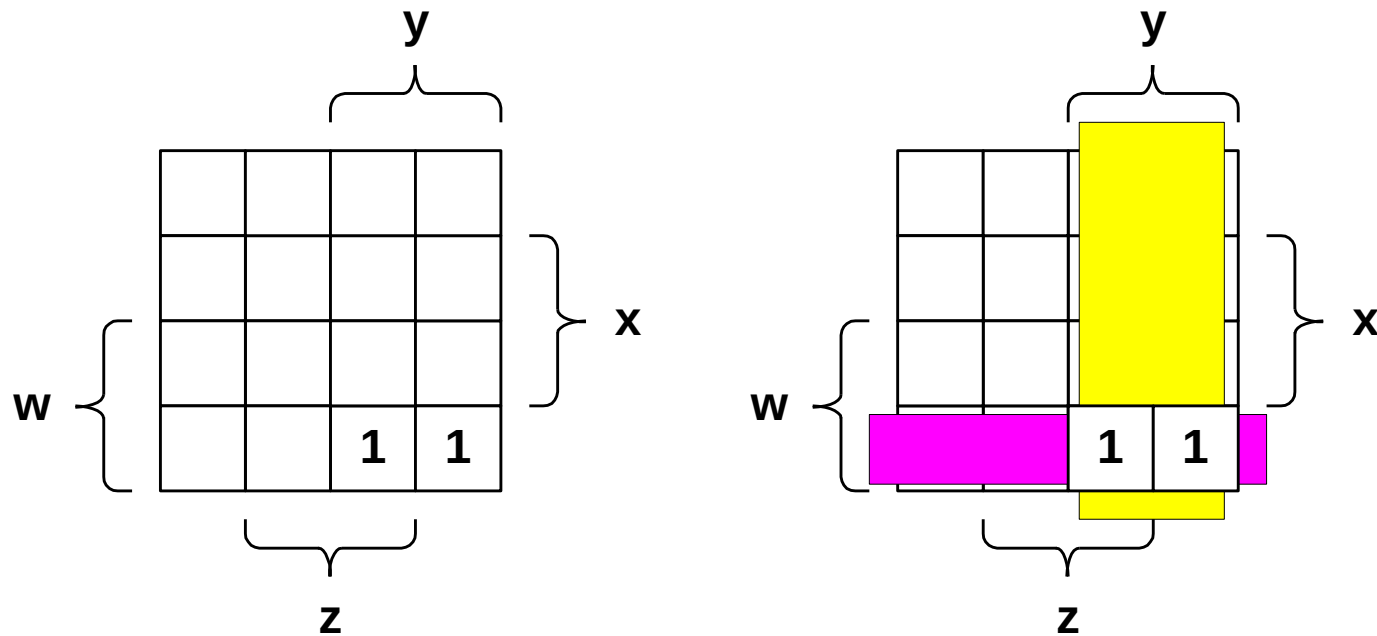
In this expression, we want to group variables into

- As **large** a group as possible
- With the **fewest** possible groups
- With each group containing **2^n cells** where $n \in \mathbb{Z}$.

4-variable Karnaugh maps

Example 1:

Find the Boolean product represented by the K-map below



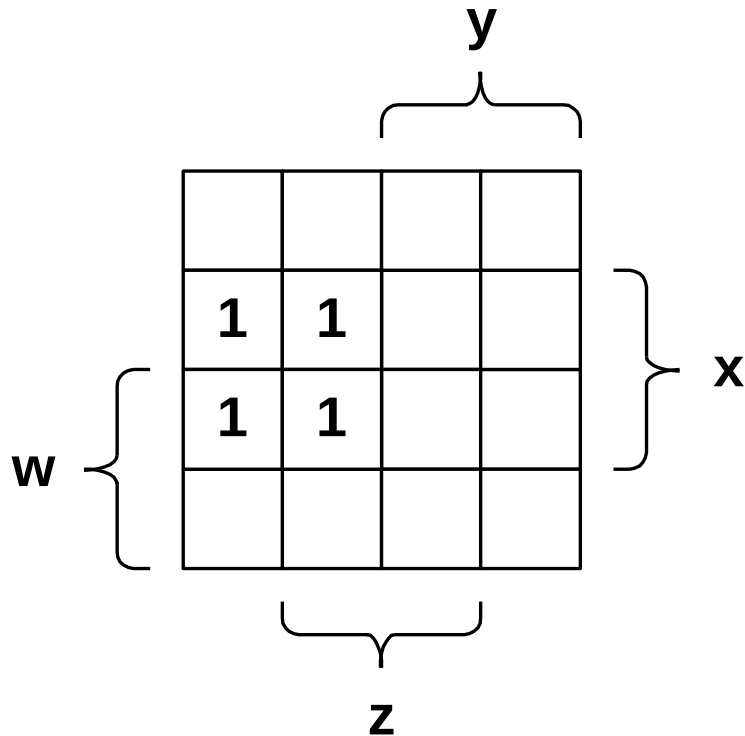
The row represents wx' and the columns represent y .

So the product is $wx'y$.

4-variable Karnaugh maps

Example 2:

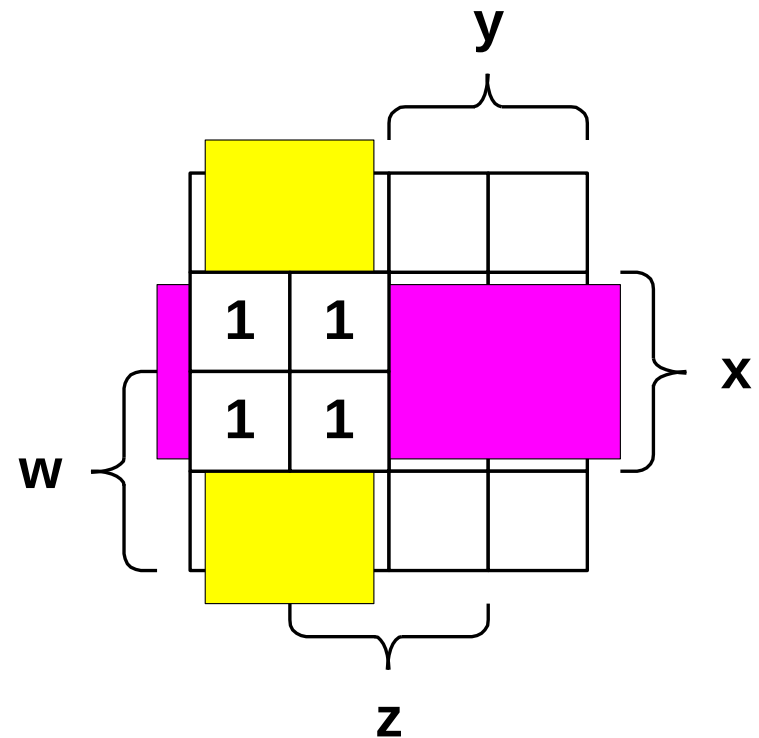
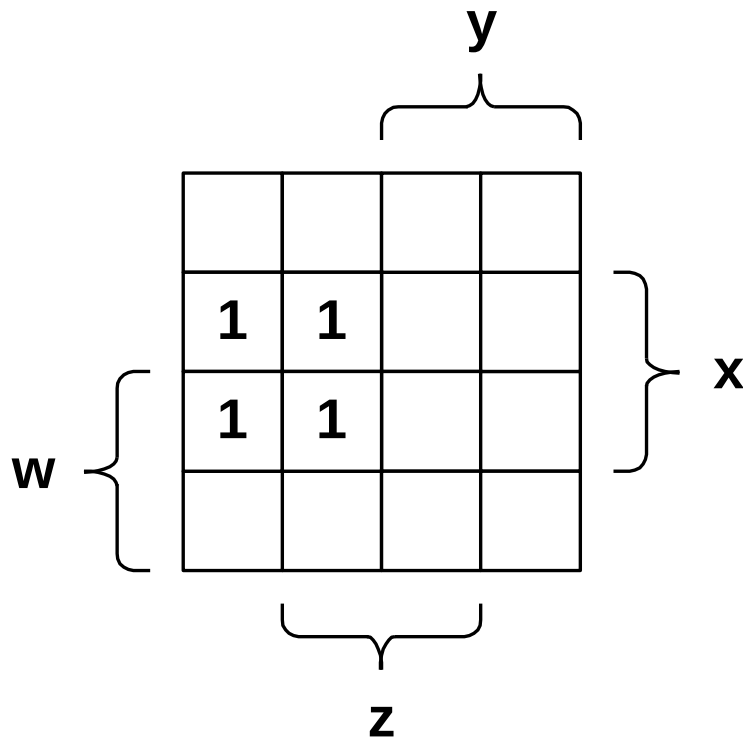
Find the Boolean product represented by the K-map below



4-variable Karnaugh maps

Example 2:

Find the Boolean product represented by the K-map below

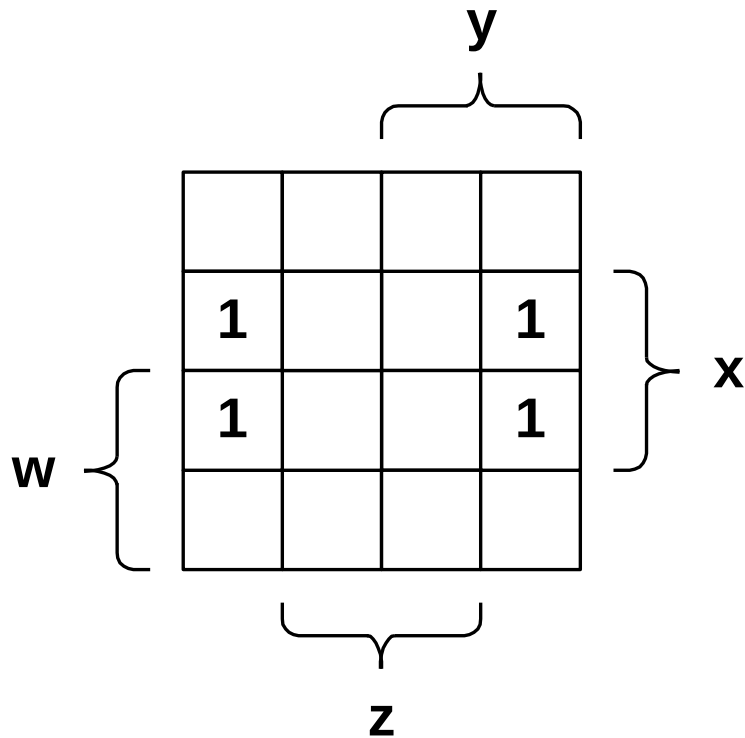


Answer: $wxy'z'$

4-variable Karnaugh maps

Example 3:

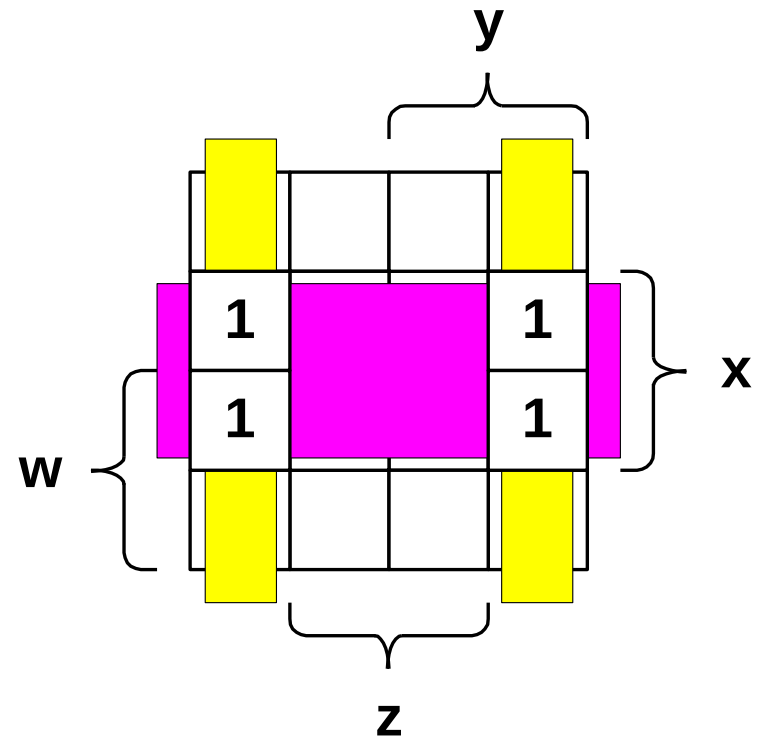
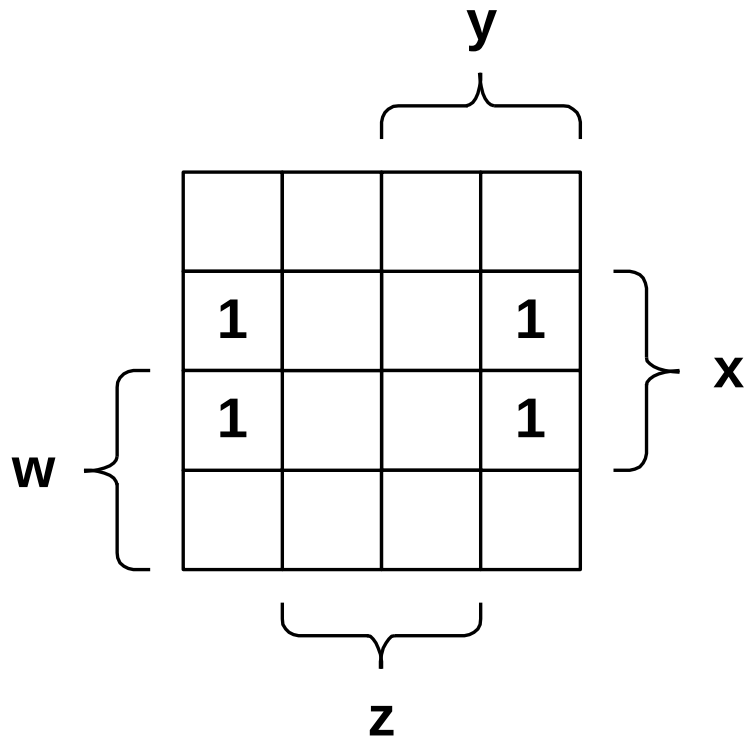
Find the Boolean product represented by the K-map below



4-variable Karnaugh maps

Example 3:

Find the Boolean product represented by the K-map below

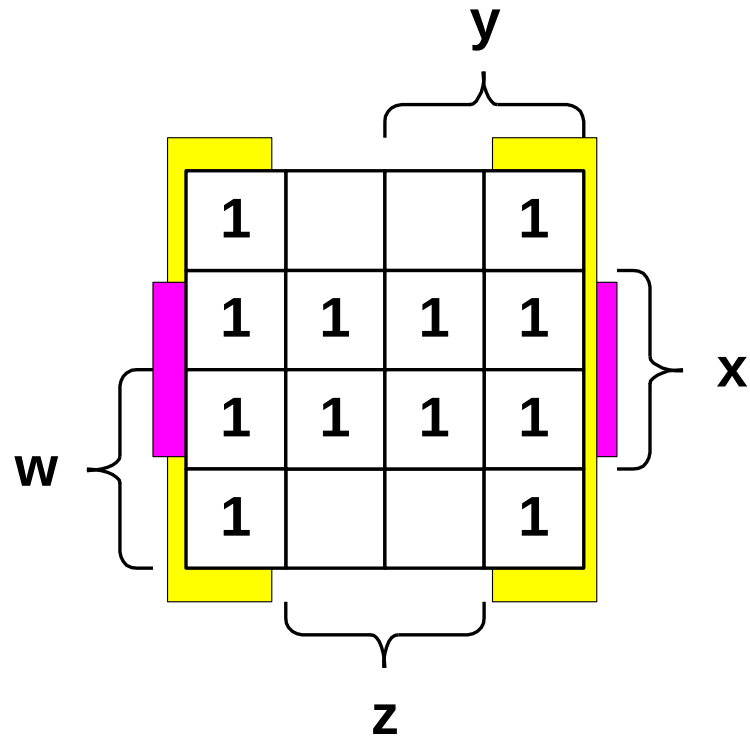


Answer: xz'

4-variable Karnaugh maps

Example 4:

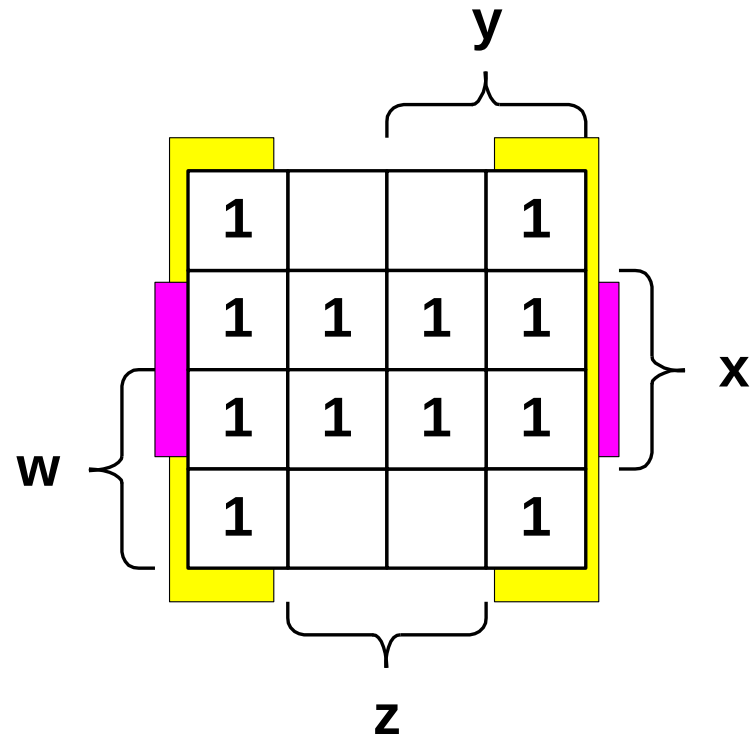
Find the Boolean **sum of products** represented by the K-map below



4-variable Karnaugh maps

Example 4:

Find the Boolean **sum of products** represented by the K-map below

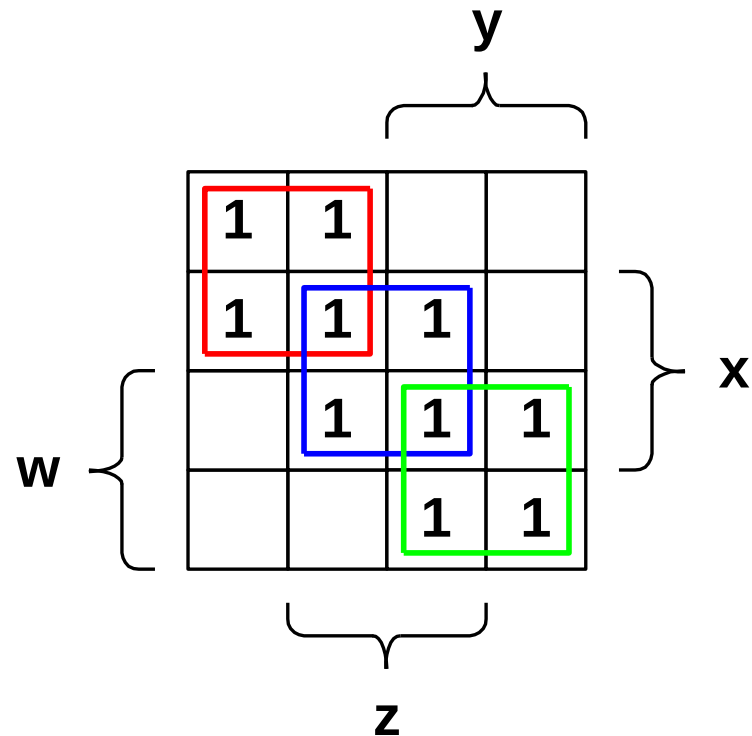


Answer: $x + z'$

4-variable Karnaugh maps

Example 5:

Find the Boolean **sum of products** represented by the K-map below

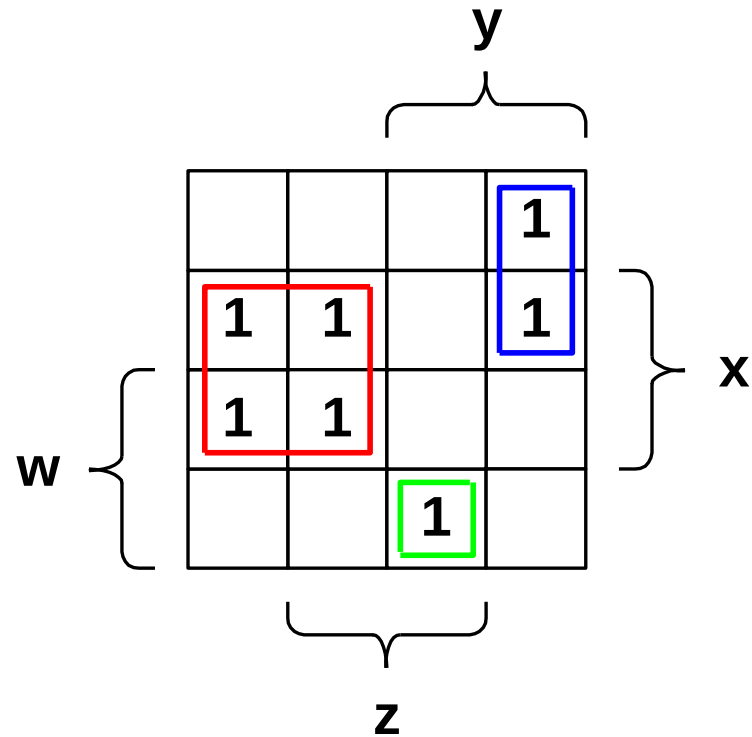


Answer: $wy + xz + w'y'$

4-variable Karnaugh maps

Example 6:

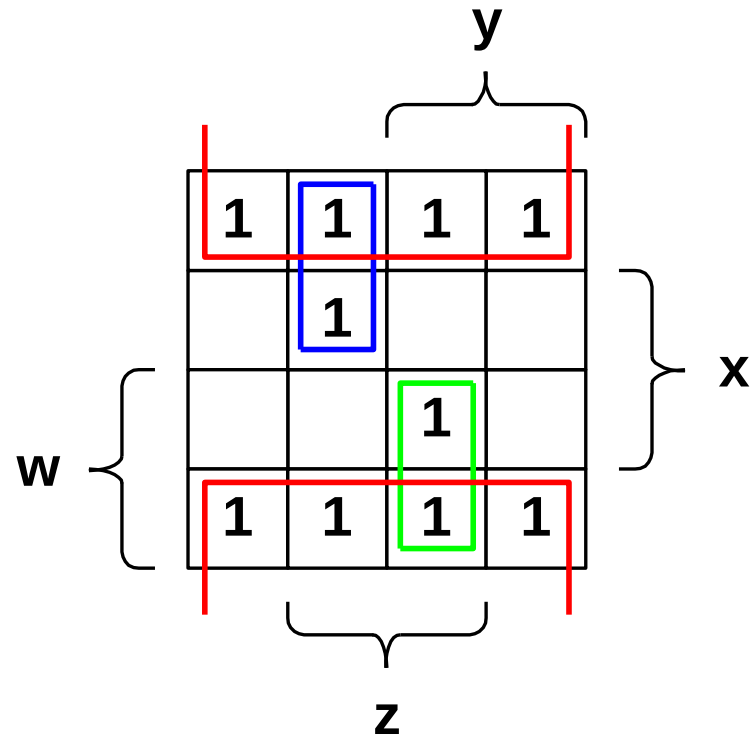
Find the Boolean sum of products represented by the K-map below



4-variable Karnaugh maps

Example 7:

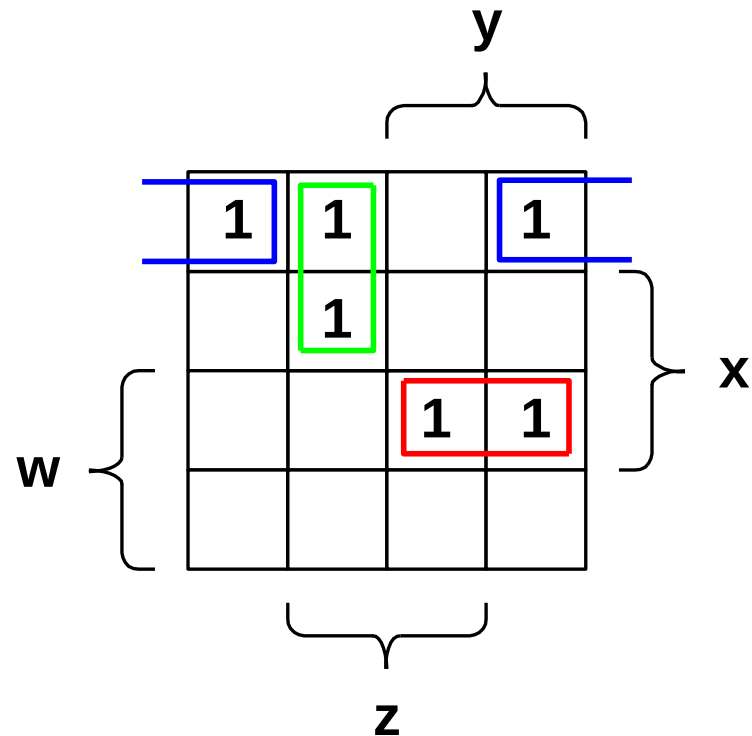
Find the Boolean sum of products represented by the K-map below



4-variable Karnaugh maps

Example 8:

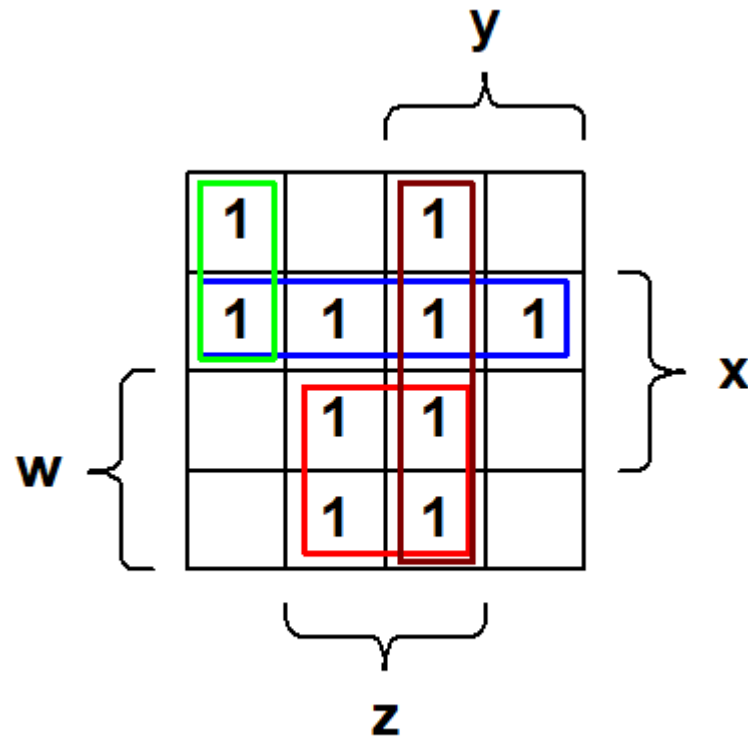
Find the Boolean sum of products represented by the K-map below



4-variable Karnaugh maps

Example 9:

Find the Boolean sum of products represented by the K-map below



Minimal sum of products

Our aim is to find the **simplest possible expression** that is still a **sum of products**.

We want a sum of products with the **fewest terms** and **with the fewest variables**.

Such an expression is called a **minimal sum of products** (Min SOP), and the corresponding circuit is called a **minimal AND-OR circuit**.

Minimal sum of products

For example, the expression

$$xyz + xyz' + xy'z$$

is a sum of products, and it can be simplified via

$$\begin{aligned}xyz + xyz' + xy'z &= xyz + xyz' + xy'z + xyz \\&= xy(z + z') + xz(y + y') \\&= xy + xz\end{aligned}$$

And $xy + xz$ is a Min SOP.

Notice that we will **not** simplify further to $x(y + z)$ because this is no longer a SOP.

SOPs, CSOPs and Min SOPs

A **SOP** is simply an expression consisting of **products of variables** (with or without complements) added together.

A **CSOP** is an expression that has been expanded into its **fundamental products**. (Uniquely defined.)

A **Min SOP** is an expression that has been simplified so as to consist of the **fewest possible terms and variables**, and **still remains a SOP**. (Not uniquely defined.)

So, at one end of the spectrum we have a CSOP, and at the other end we have a Min SOP.

SOPs, CSOPs and Min SOPs

For example, consider the expression $wyz + w'yz + w'xyz'$.

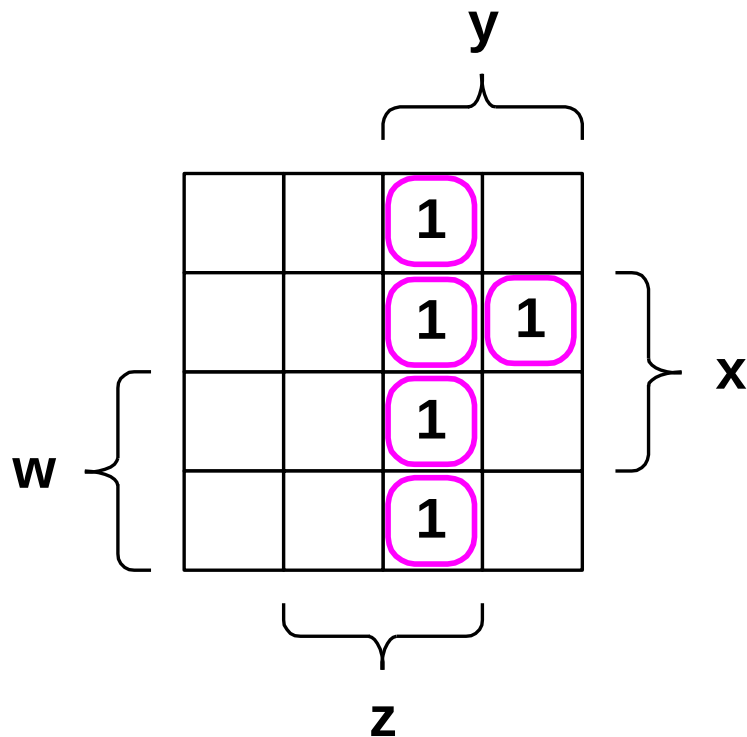
(i) We can expand it to

$$\begin{aligned} & w(x+x')yz + w'(x+x')yz + w'xyz' \\ &= wxyz + wx'yz + w'xyz + w'x'yz + w'xyz' \quad (\text{CSOP}) \end{aligned}$$

(ii) Or we can simplify it to

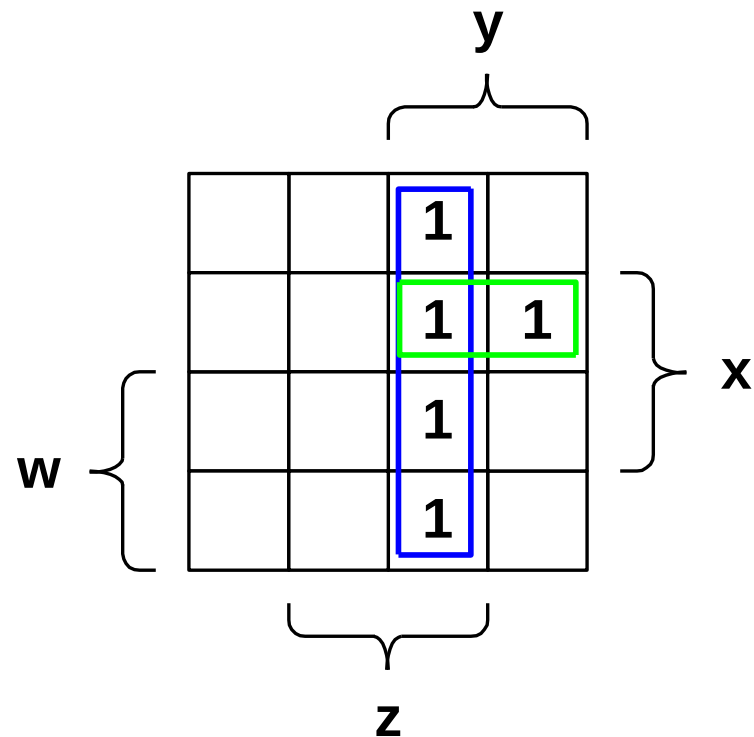
$$\begin{aligned} & (w+w')yz + w'xyz' = yz + w'xyz' \\ &= (1+w'x)yz + w'xyz' = yz + w'xyz + w'xyz' = yz + (w'xy)(z+z') \\ &= yz + w'xy \quad (\text{Min SOP}) \end{aligned}$$

SOPs, CSOPs and Min SOPs



5 fundamental products

$$wxyz + wx'yz + w'xyz + w'x'yz + w'xy$$

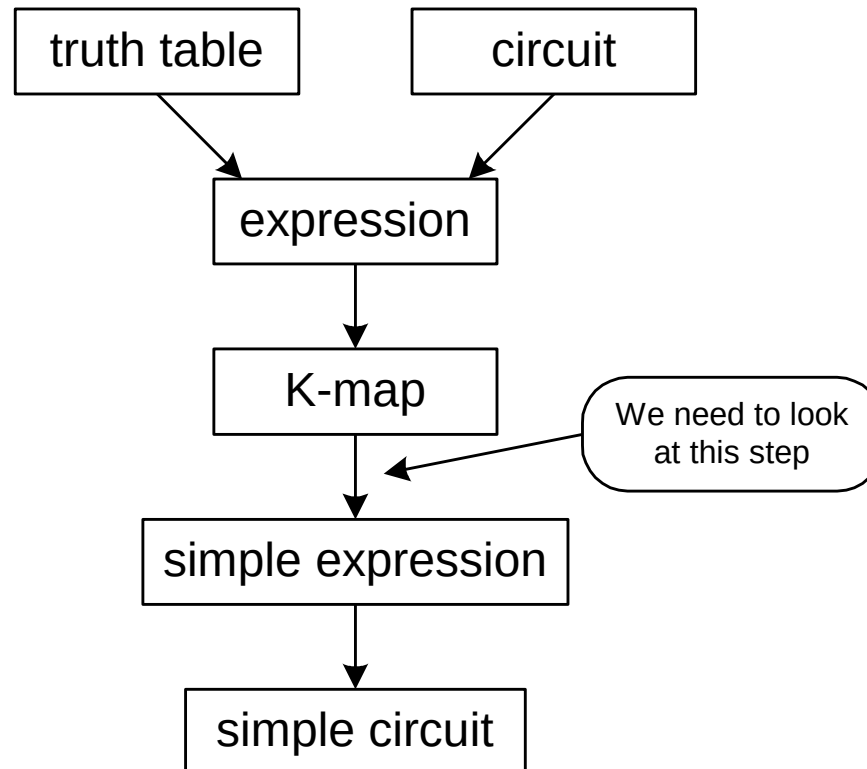


Two groups $yz + w'xy$

Typical design problems

We are concerned with two types of problems:

- Given the specifications for a circuit in the form of a truth table **design the simplest circuit** to meet the specifications.
- Given a circuit, design a **simpler** circuit that performs the same way.



Finding Min SOPs from K-maps

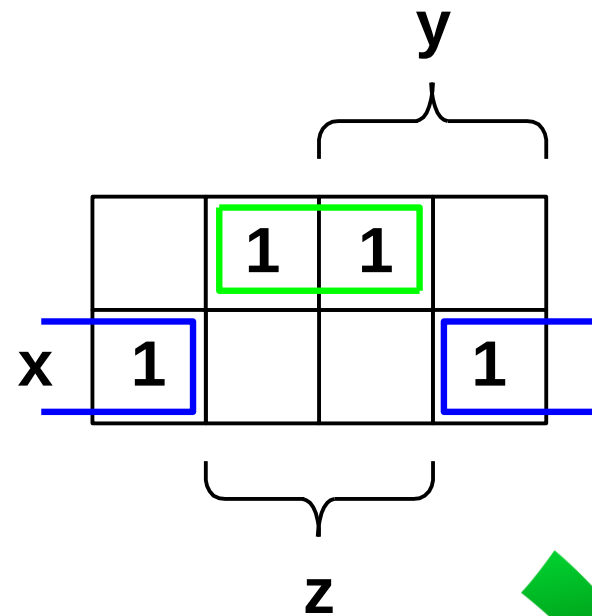
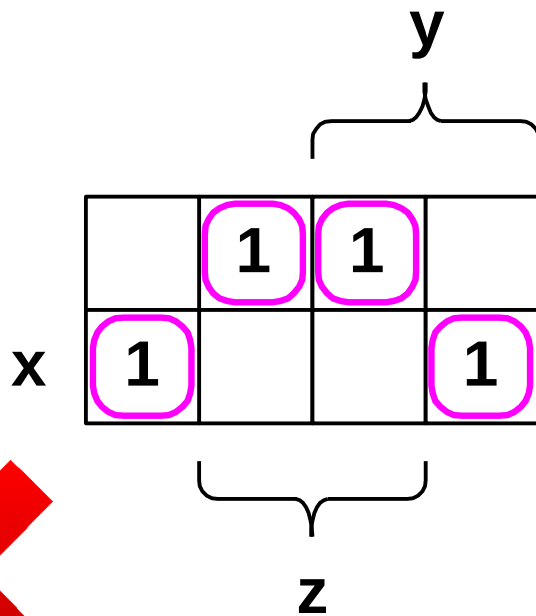
Given a K-map we need to be able to find the corresponding Min SOP. To do this we look for groups of cells that correspond to **products with as few variables as possible**.

So remember:

- Look for groups that are 4 x 2, 4 x 1, 2 x 2, 2 x 1
(because products involve only powers of 2).
- Make groups as large as possible
(the larger the group, the simpler the product).
- Overlap groups if possible
(without introducing redundancy).
- Don't forget the "split groups"
(with cells at opposite edges of the K-map).

Finding Min SOPs from K-maps

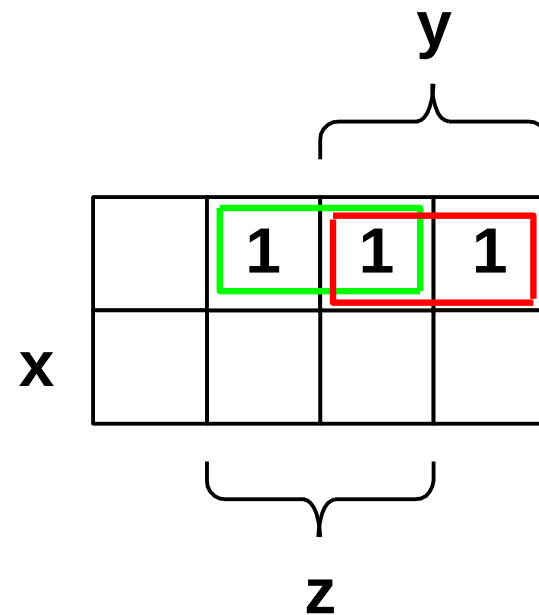
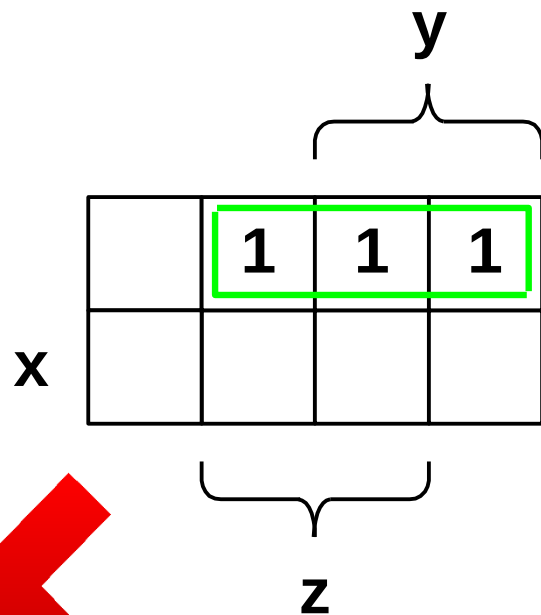
Examples of incorrect and correct grouping of cells



Make groups as large as possible

Finding Min SOPs from K-maps

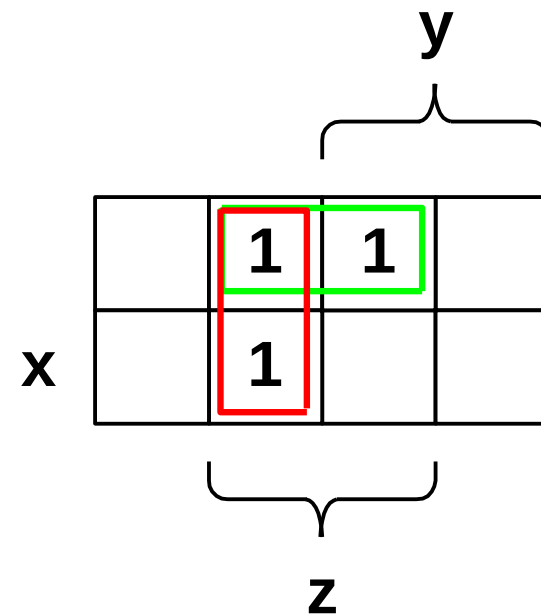
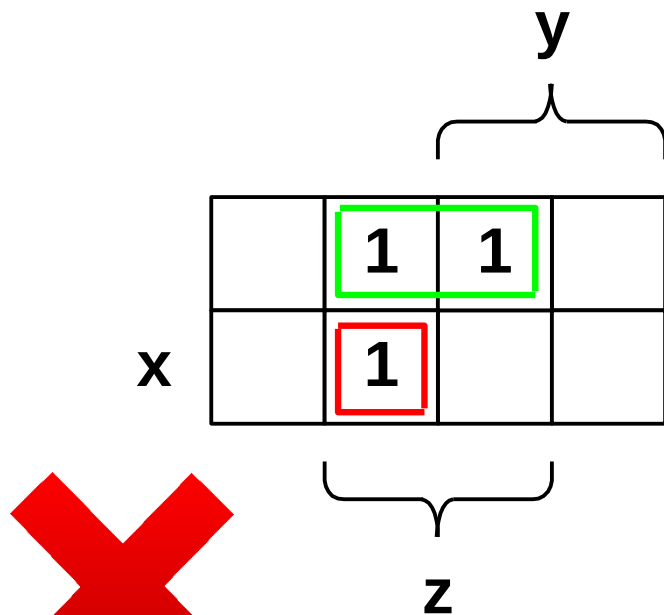
Examples of incorrect and correct grouping of cells



Look for powers of 2
(The first grouping is impossible!)

Finding Min SOPs from K-maps

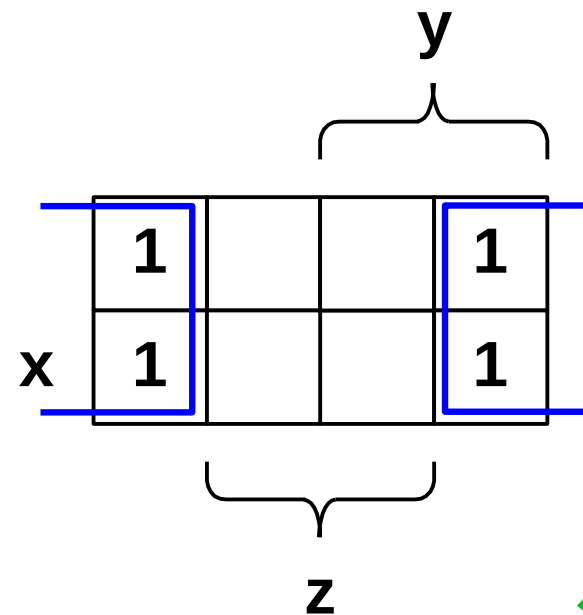
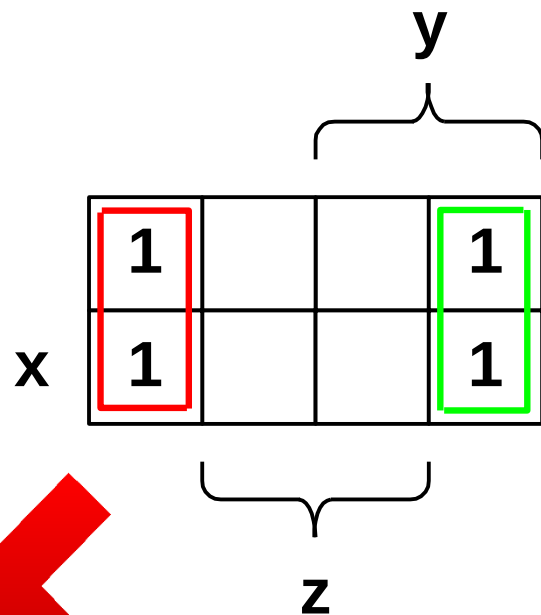
Examples of incorrect and correct grouping of cells



Overlap if possible

Finding Min SOPs from K-maps

Examples of incorrect and correct grouping of cells

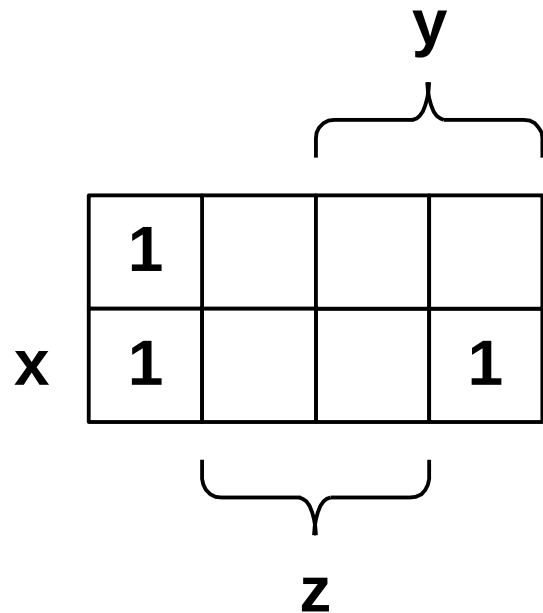


Don't forget the "split groups"

Examples of finding Min SOPs from K-maps

Example 1:

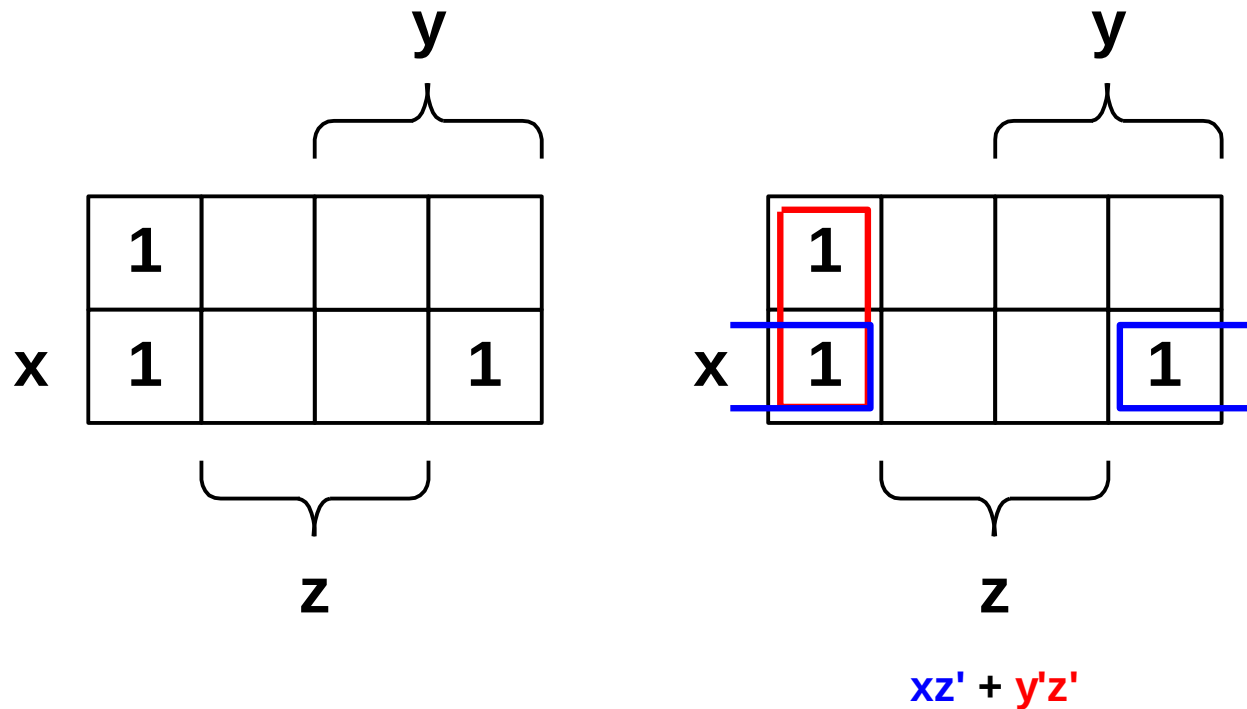
Find the Min SOP corresponding to the Karnaugh map:



Examples of finding Min SOPs from K-maps

Example 1:

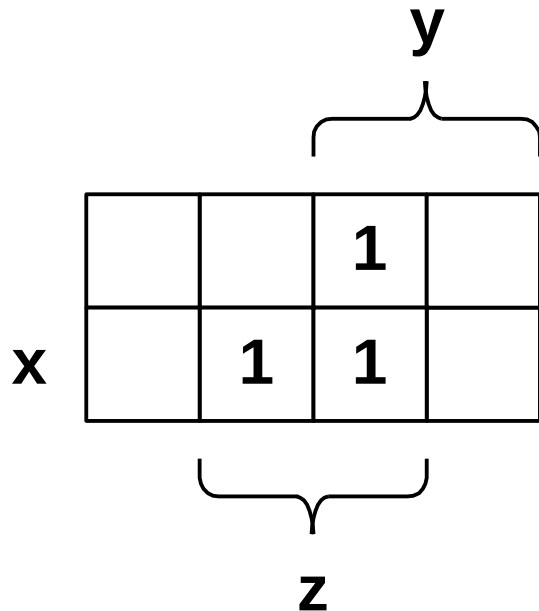
Find the Min SOP corresponding to the Karnaugh map:



Examples of finding Min SOPs from K-maps

Example 2:

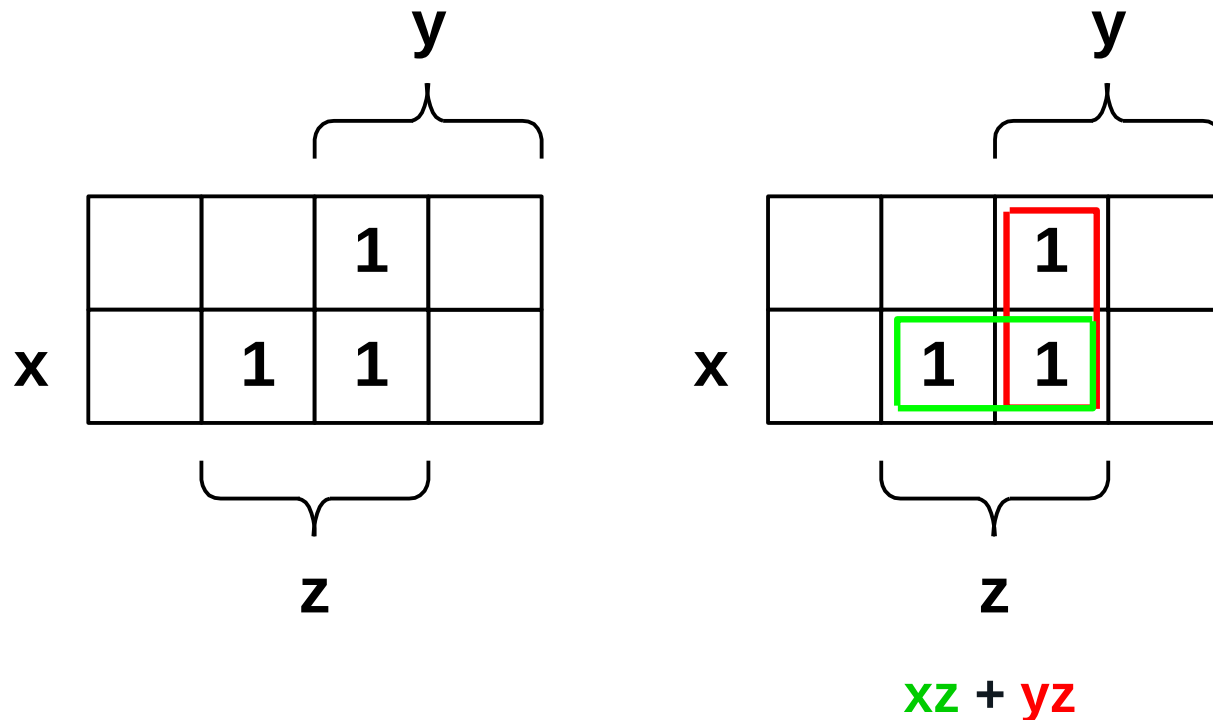
Find the Min SOP corresponding to the Karnaugh map:



Examples of finding Min SOPs from K-maps

Example 2:

Find the Min SOP corresponding to the Karnaugh map:



Examples of finding Min SOPs from K-maps

Example 3:

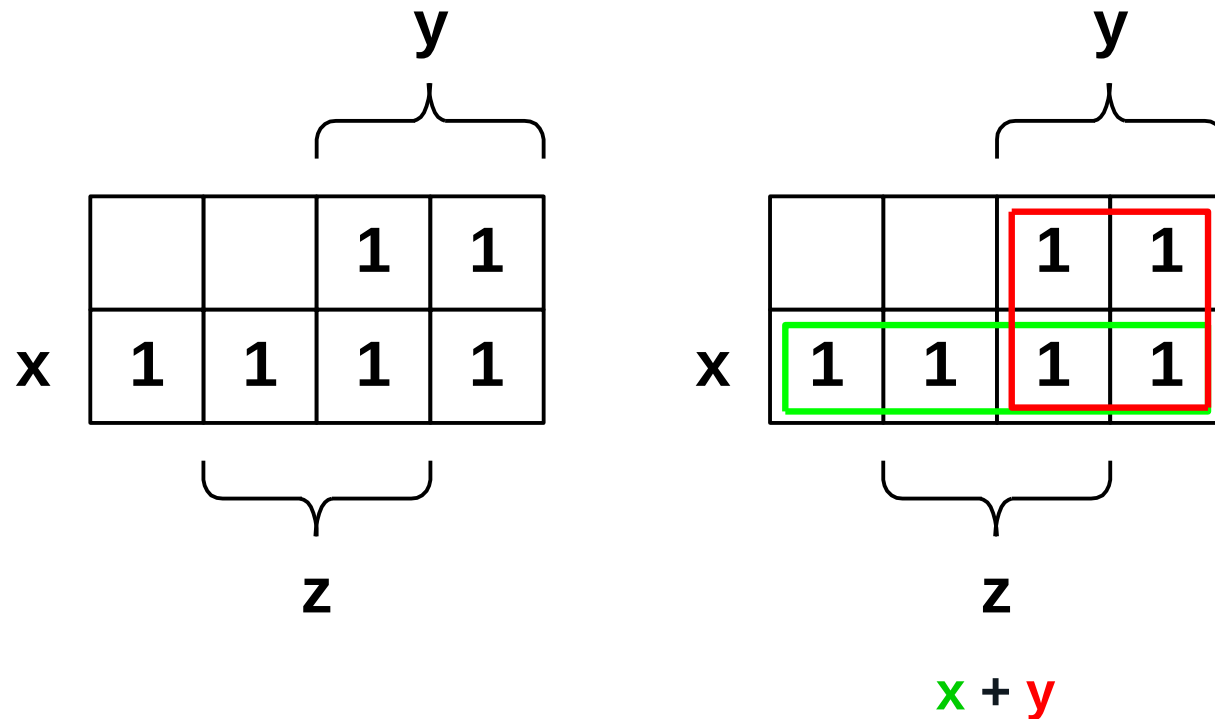
Find the Min SOP corresponding to the Karnaugh map:

		1	1
1	1	1	1

Examples of finding Min SOPs from K-maps

Example 3:

Find the Min SOP corresponding to the Karnaugh map:



Examples of finding Min SOPs from K-maps

Example 4:

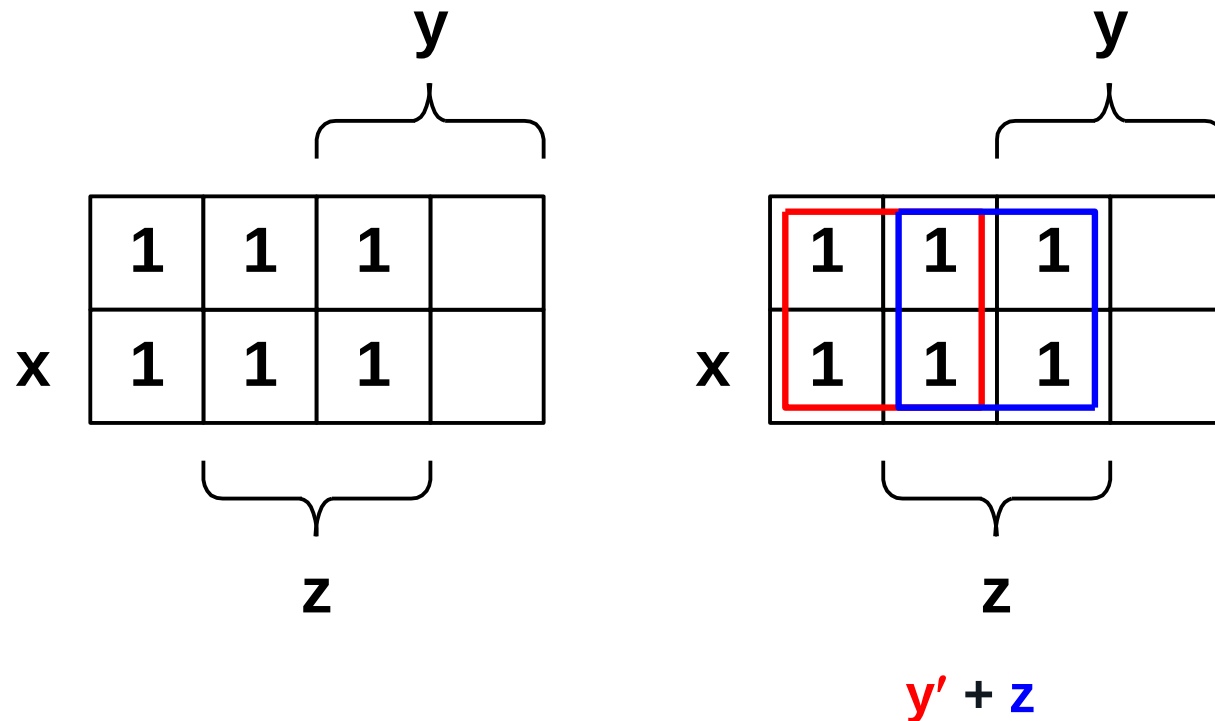
Find the Min SOP corresponding to the Karnaugh map:

	y			
				
x	1	1	1	
	1	1	1	
				
	z			

Examples of finding Min SOPs from K-maps

Example 4:

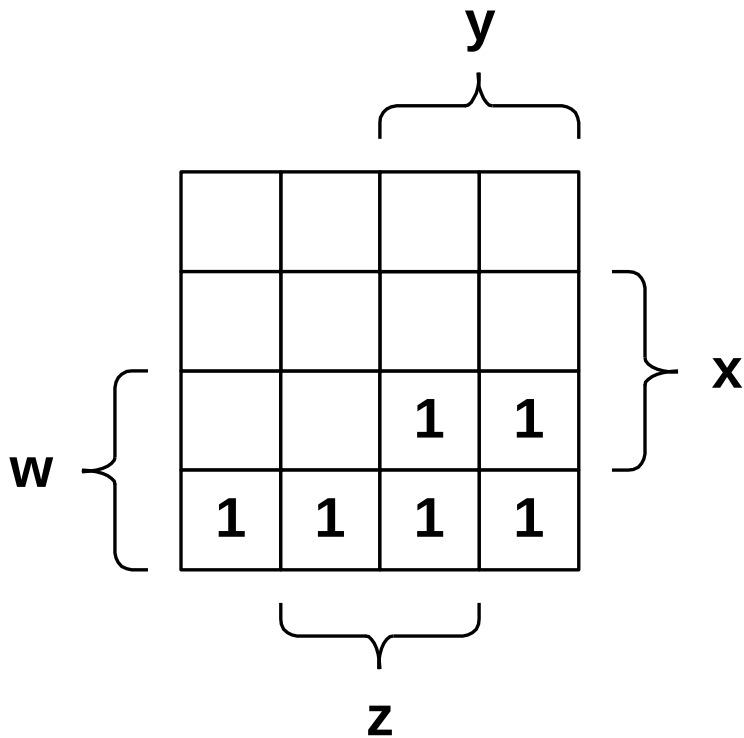
Find the Min SOP corresponding to the Karnaugh map:



Examples of finding Min SOPs from K-maps

Example 5:

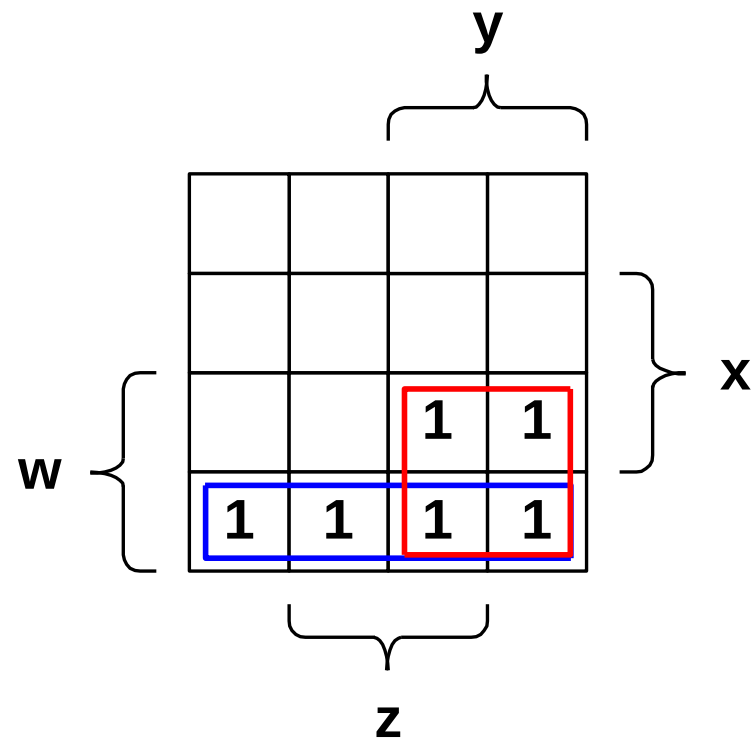
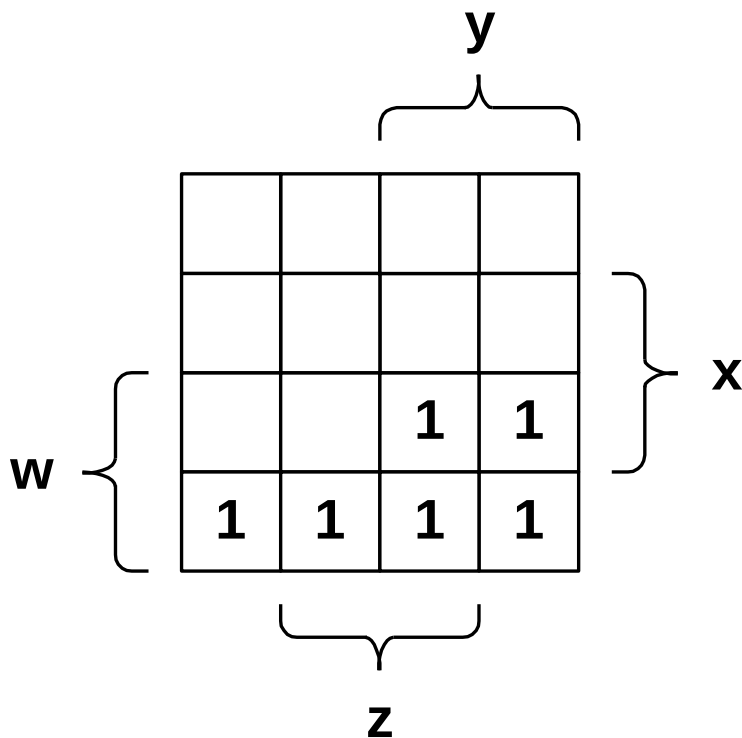
Find the Min SOP corresponding to the Karnaugh map:



Examples of finding Min SOPs from K-maps

Example 5:

Find the Min SOP corresponding to the Karnaugh map:

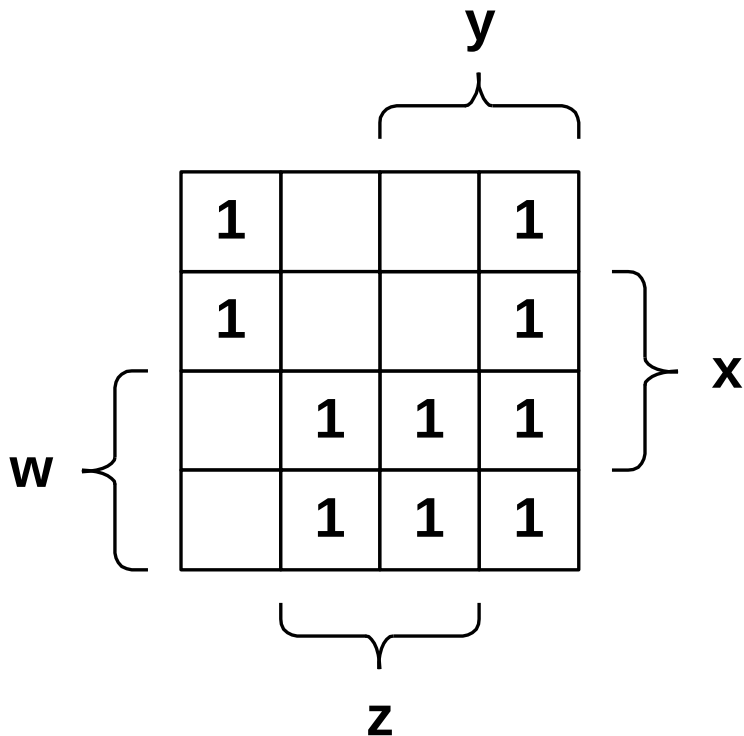


$$wy + wx'$$

Examples of finding Min SOPs from K-maps

Example 6:

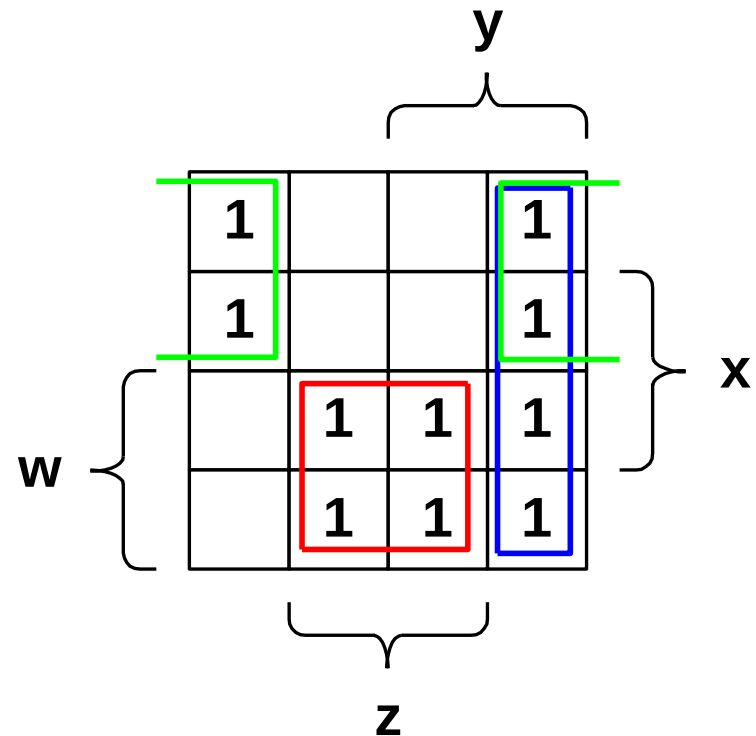
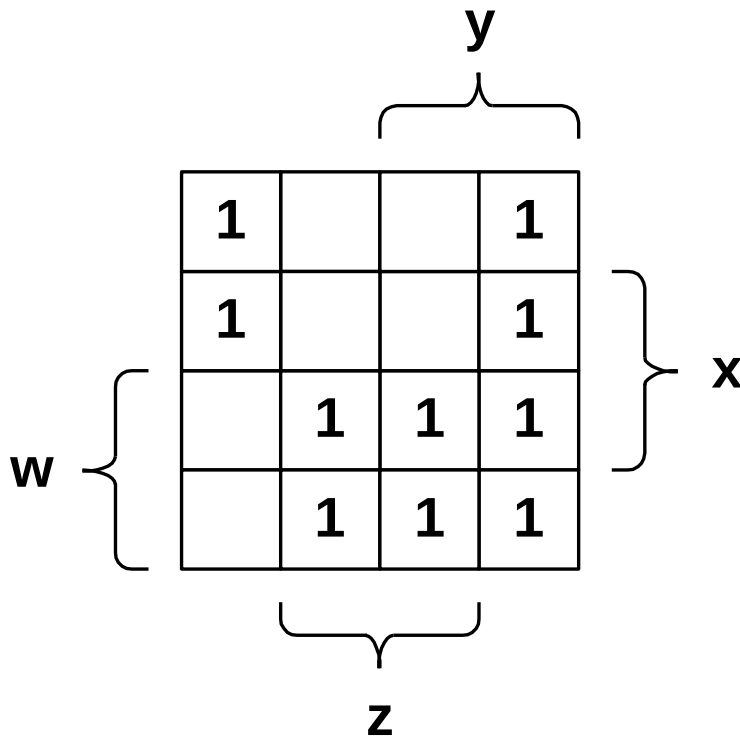
Find the Min SOP corresponding to the Karnaugh map:



Examples of finding Min SOPs from K-maps

Example 6:

Find the Min SOP corresponding to the Karnaugh map:

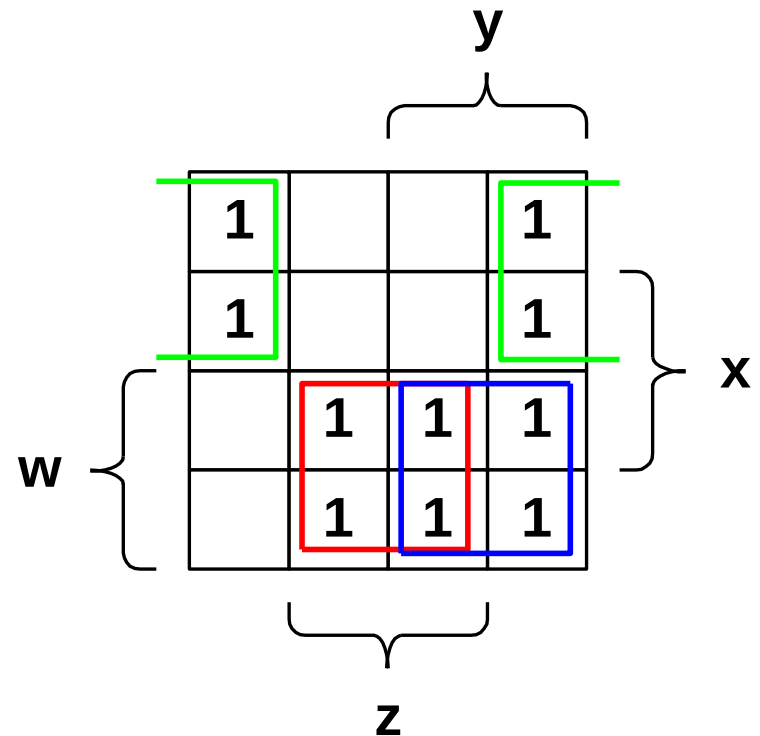
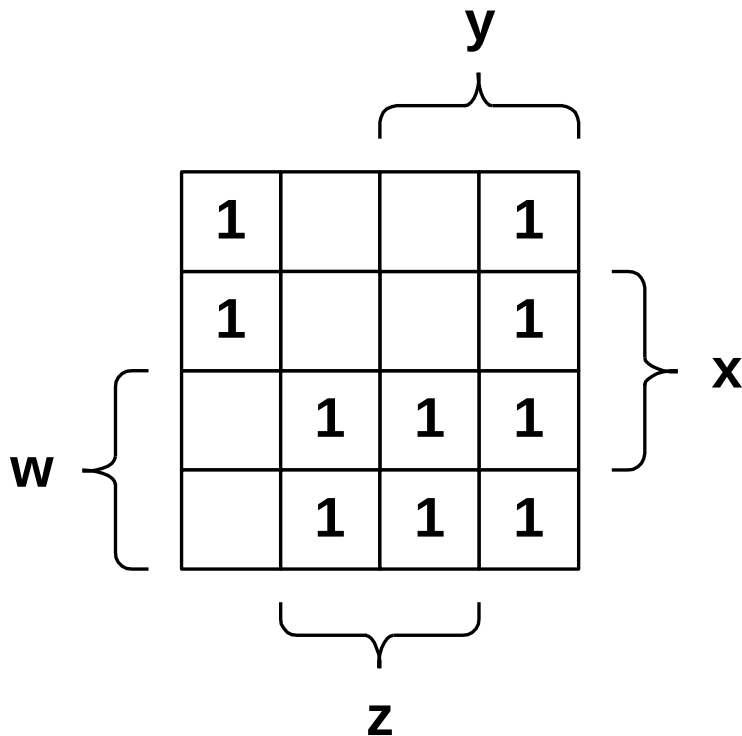


$$wz + yz' + w'z'$$

Examples of finding Min SOPs from K-maps

Example 6:

Find the Min SOP corresponding to the Karnaugh map:

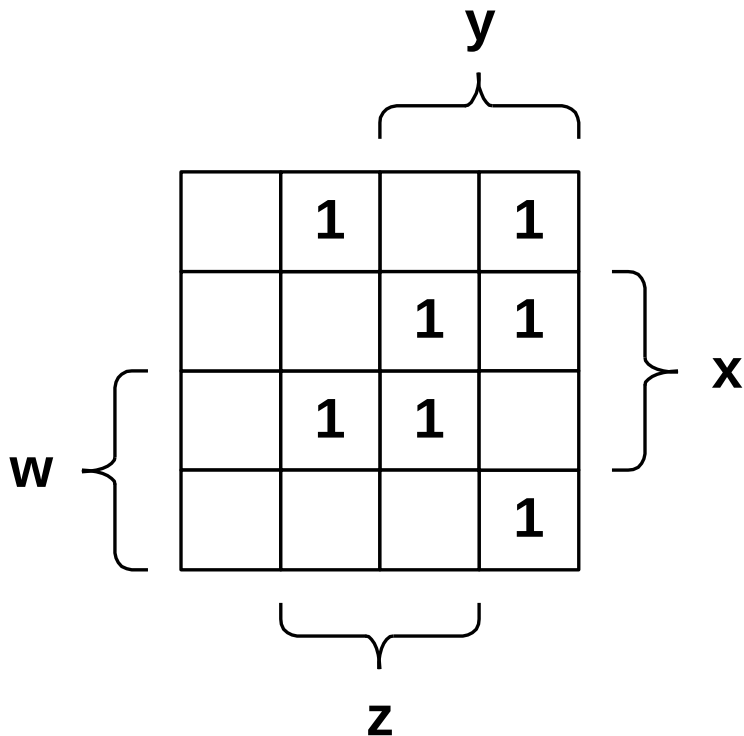


$$wz + wy + w'z'$$

Examples of finding Min SOPs from K-maps

Example 7:

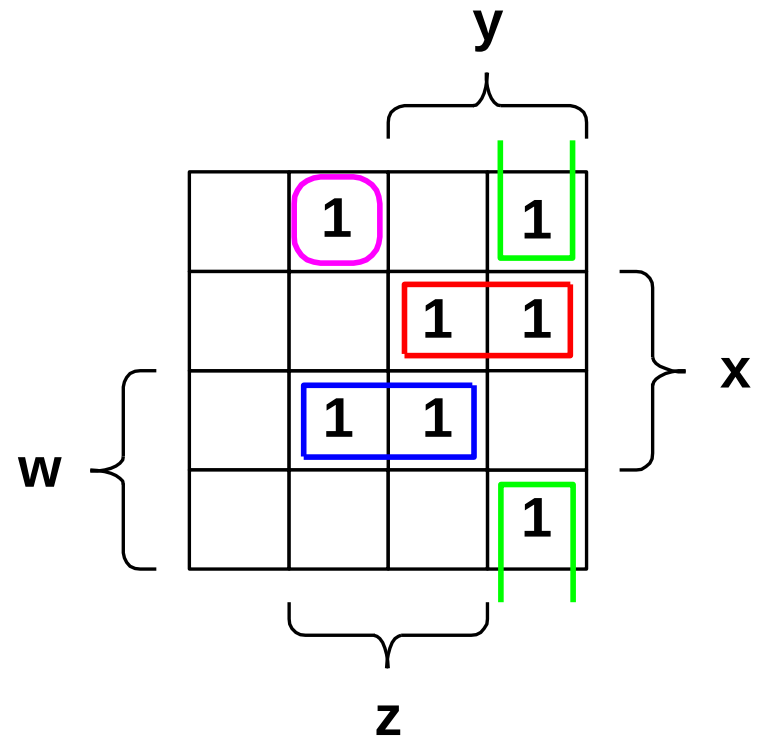
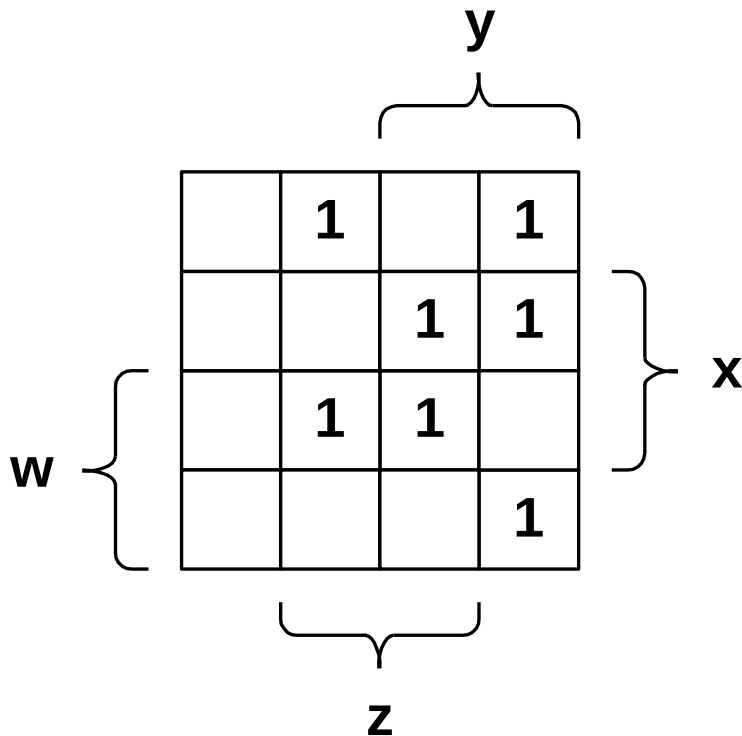
Find the Min SOP corresponding to the Karnaugh map:



Examples of finding Min SOPs from K-maps

Example 7:

Find the Min SOP corresponding to the Karnaugh map:

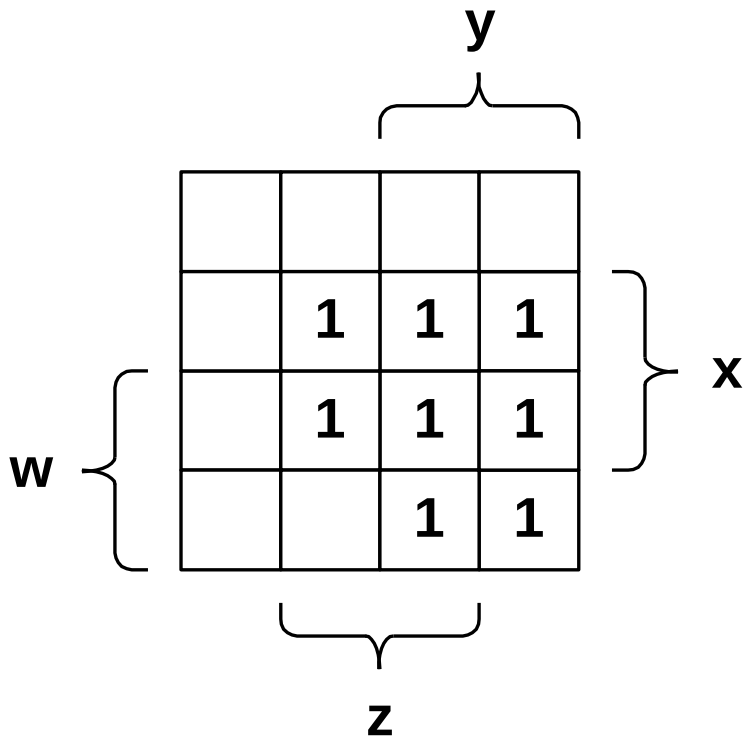


$$wxz + w'xy + x'yz' + w'x'y'z$$

Examples of finding Min SOPs from K-maps

Example 8:

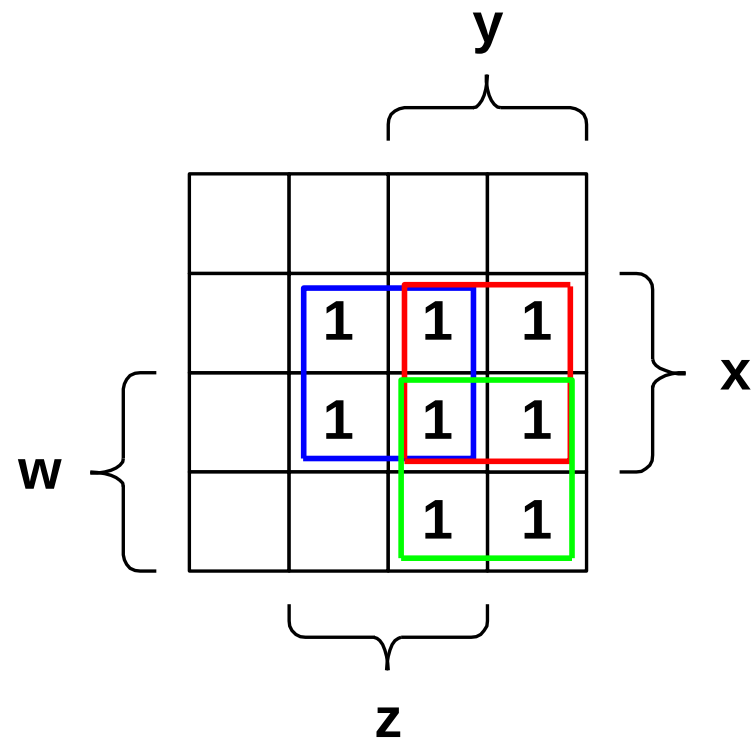
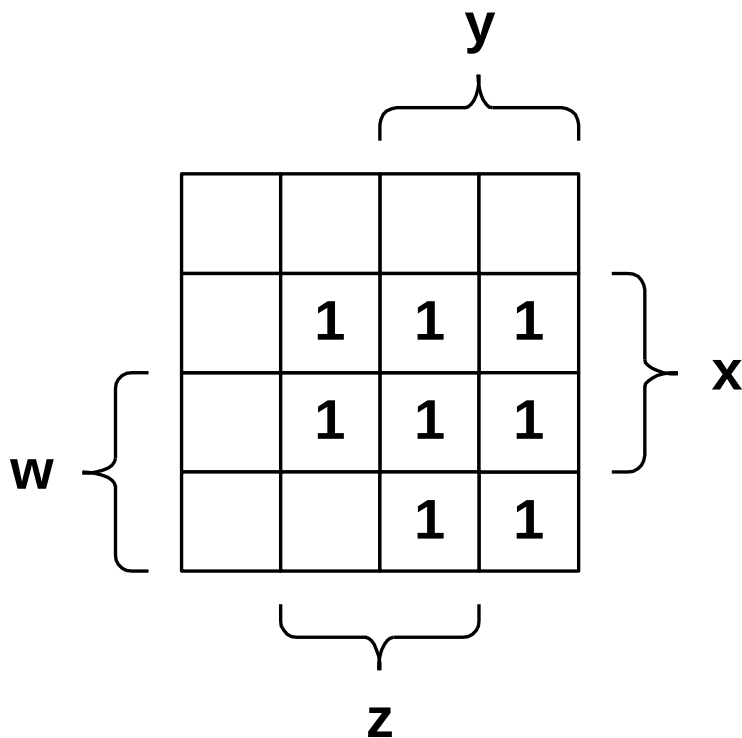
Find the Min SOP corresponding to the Karnaugh map:



Examples of finding Min SOPs from K-maps

Example 8:

Find the Min SOP corresponding to the Karnaugh map:

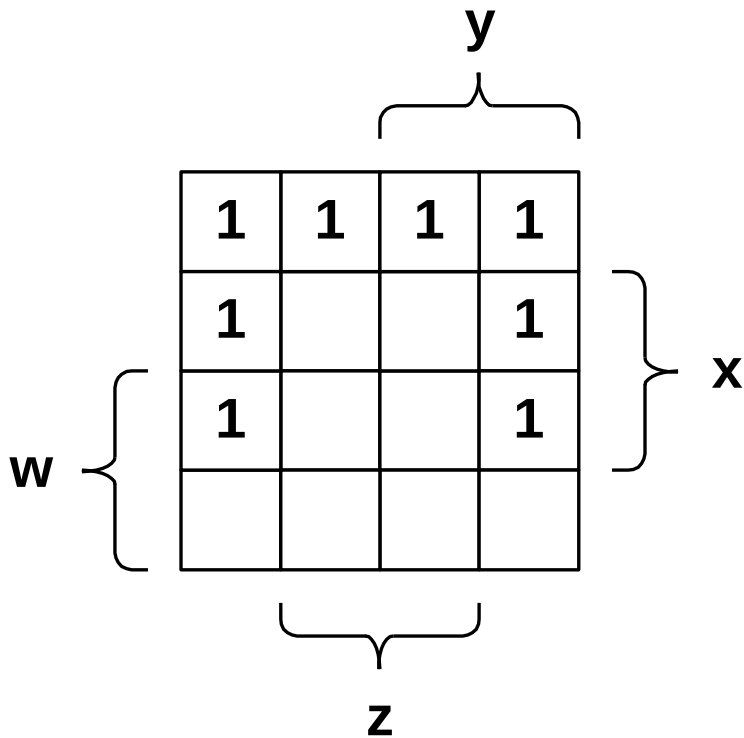


$$xz + xy + wy$$

Examples of finding Min SOPs from K-maps

Example 9:

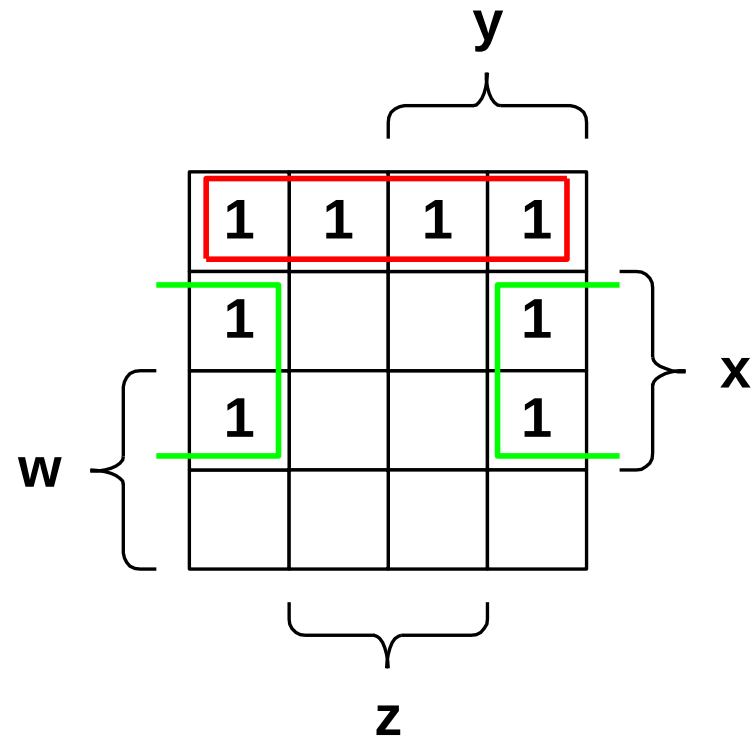
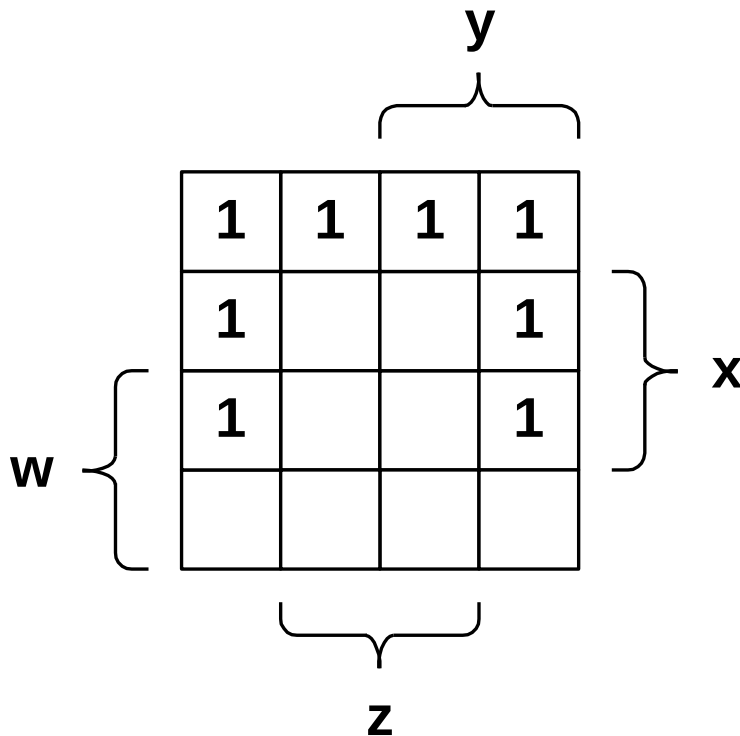
Find the Min SOP corresponding to the Karnaugh map:



Examples of finding Min SOPs from K-maps

Example 9:

Find the Min SOP corresponding to the Karnaugh map:

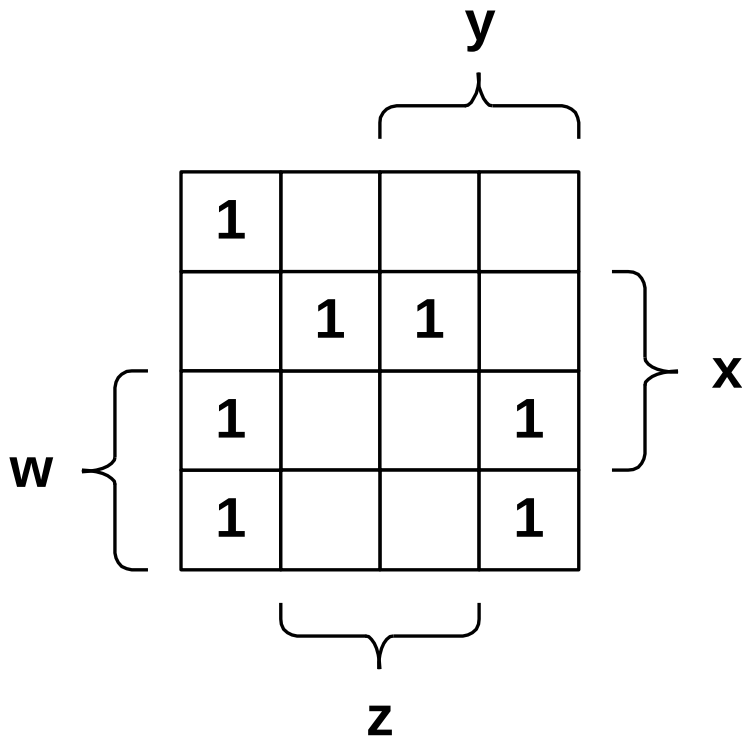


$$w'x' + xz'$$

Examples of finding Min SOPs from K-maps

Example 10:

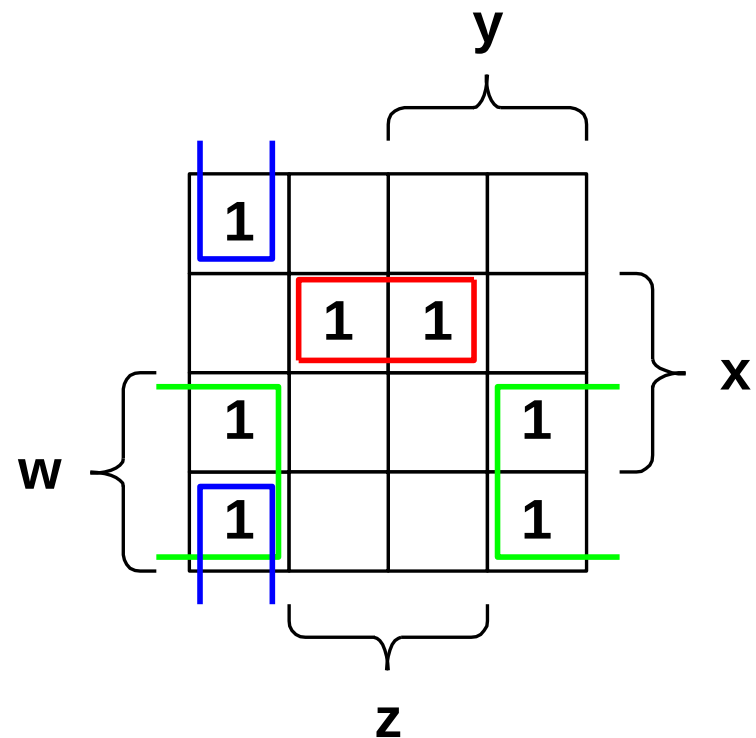
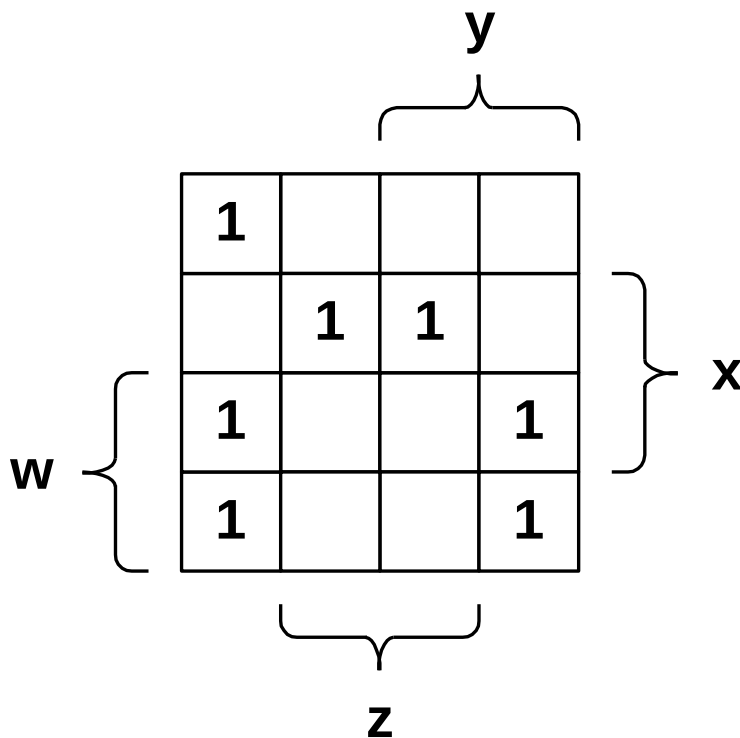
Find the Min SOP corresponding to the Karnaugh map:



Examples of finding Min SOPs from K-maps

Example 10:

Find the Min SOP corresponding to the Karnaugh map:



$$w'xz + wz' + x'y'z'$$

Design Problem Type 1: From truth table to circuit

The specifications of a circuit are given in a **truth table**. The task is to design a circuit with **output as specified in the truth table**.

The steps are:

1. from the truth table obtain a Boolean expression;
2. construct a Karnaugh map;
3. find a corresponding Min SOP;
4. draw the minimal AND-OR circuit.

Design Problem Type 1: From truth table to circuit

Example

Design a minimal AND-OR circuit to give the output P as shown in the truth table:

x	y	z	P
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

Design Problem Type 1: From truth table to circuit

Example

Design a minimal AND-OR circuit to give the output P as shown in the truth table:

x	y	z	P
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

Solution:

From the truth table, the CSOP is:

$$P = x'y'z + x'yz' + x'yz + xy'z' + xyz'$$

Design Problem Type 1: From truth table to circuit

Example

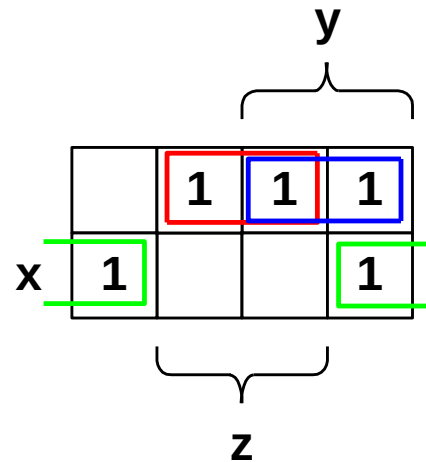
Design a minimal AND-OR circuit to give the output P as shown in the truth table:

x	y	z	P
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

Solution:

From the truth table, the CSOP is:

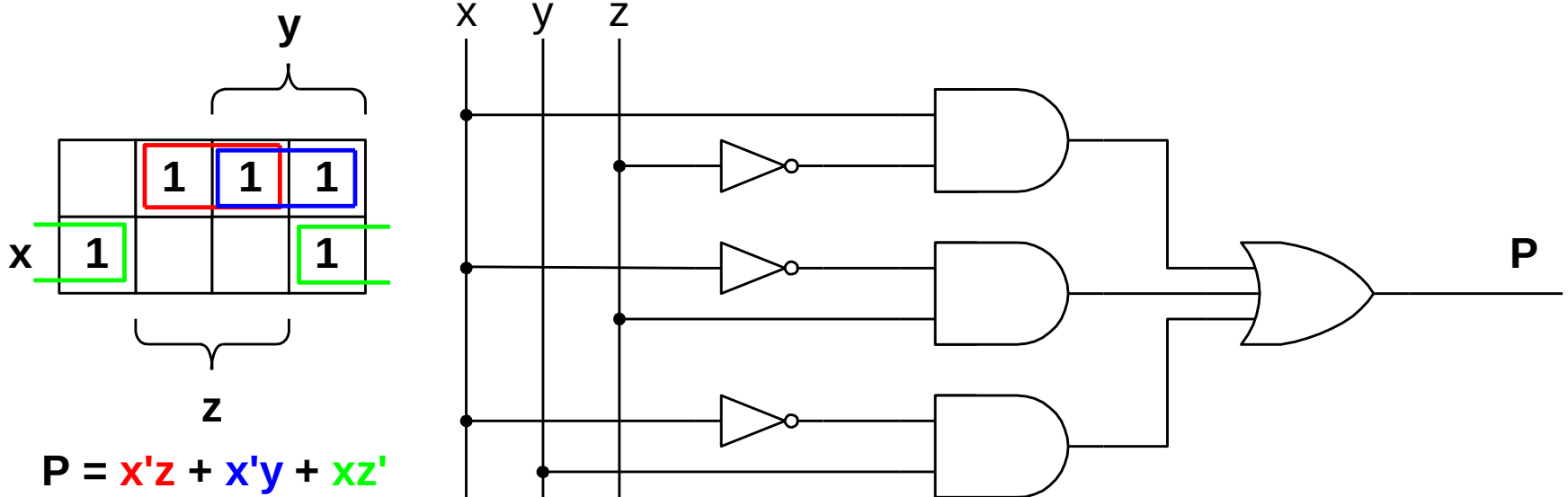
$$P = x'y'z + x'yz' + x'yz + xy'z' + xyz'$$



$$P = x'z + x'y + xz'$$

Design Problem Type 1: From truth table to circuit

Example (continued)



Design Problem Type 2: Simplifying a given circuit

A logical circuit is given, and the task is to **produce a simpler circuit** with the same output.

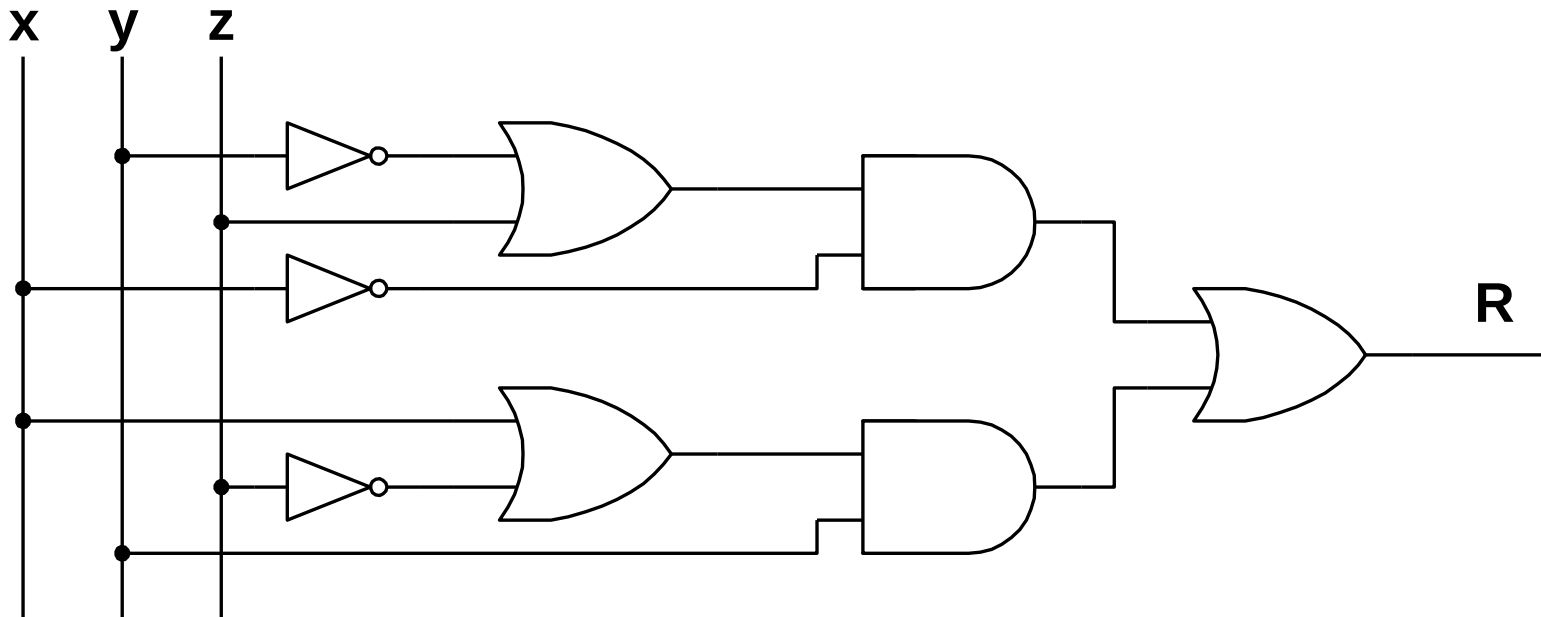
The steps are:

1. from the given circuit obtain a Boolean expression;
2. construct a Karnaugh map;
3. find a corresponding Min SOP;
4. draw the minimal AND-OR circuit.

Design Problem Type 2: Simplifying a given circuit

Example:

Design a minimal AND-OR circuit that does the same job as this circuit:

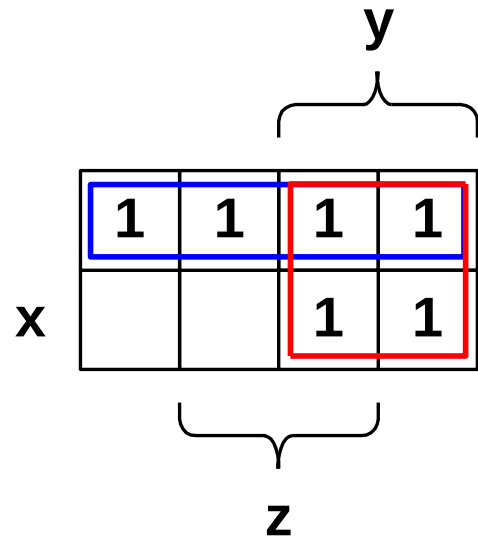


Design Problem Type 2: Simplifying a given circuit

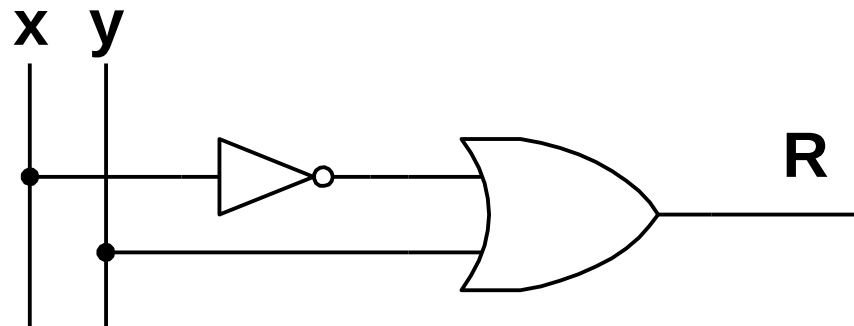
Example (solution):

From the circuit, the Boolean expression is:

$$R = (y' + z)x' + (x + z')y = x'y' + x'z + xy + yz'$$



$$R = y + x'$$



Application: 7-segment display

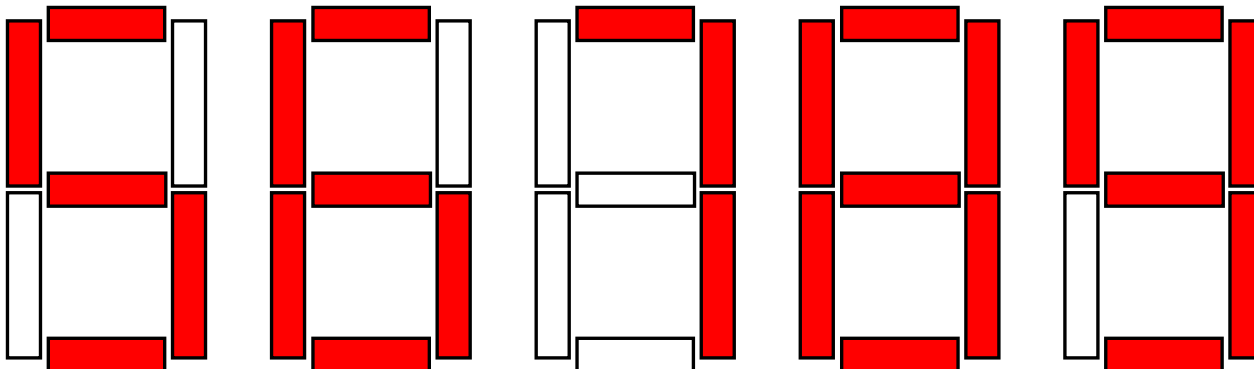
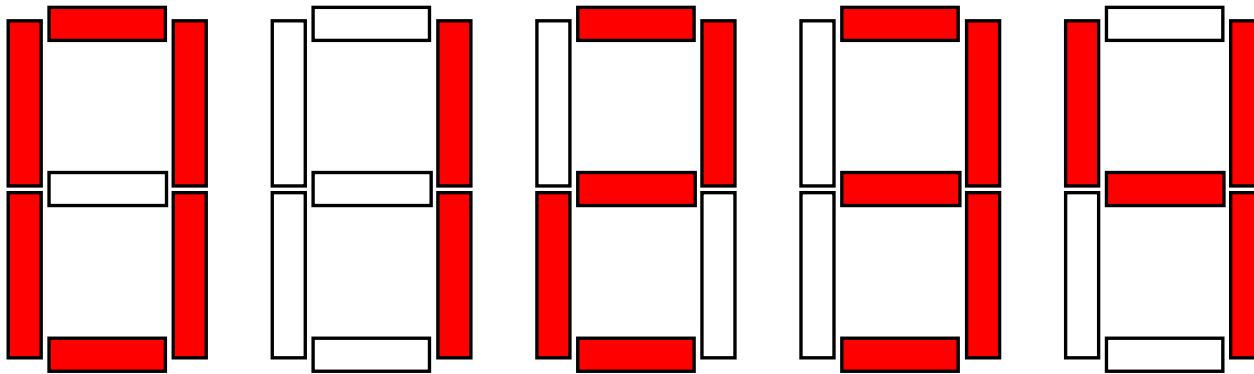
Some digital devices such as calculators, microwaves, clocks, etc. use a 7-segment display, in which each number is represented by 7 light-emitting diodes (LEDs) arranged like this:



When a numeric key (0, 1, 2, ..., 9) is pressed, the appropriate lights come on. For example if key 5 is pressed, the display is shown above.

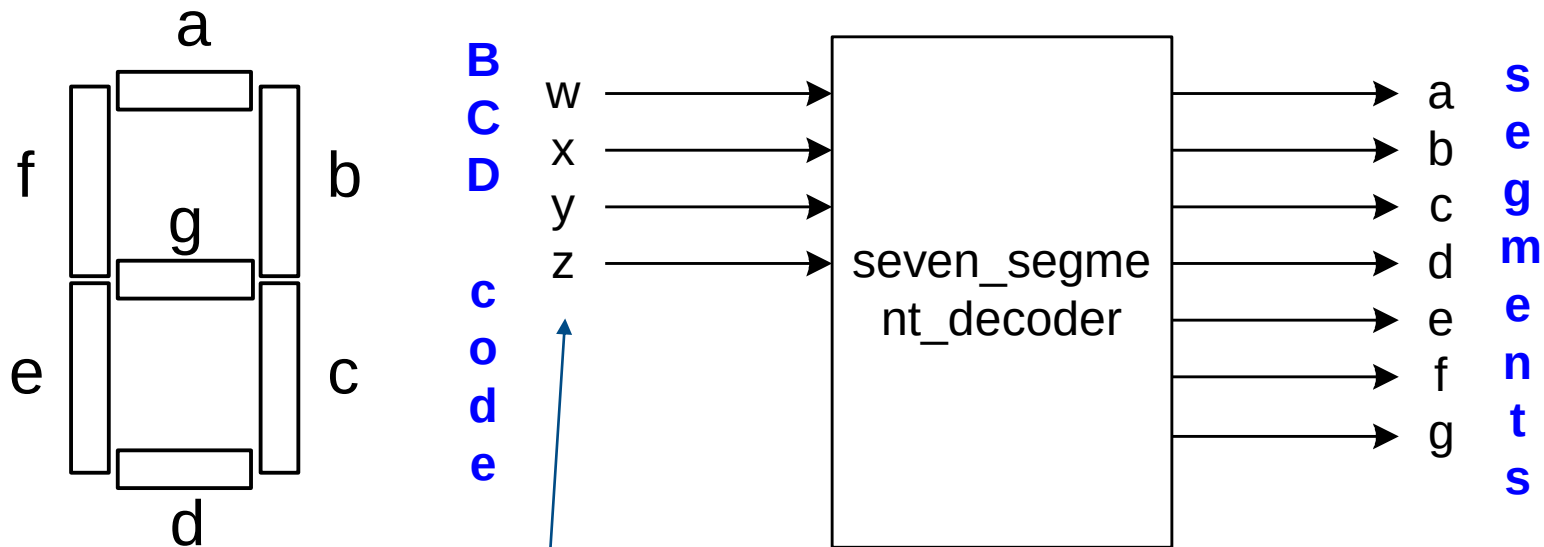
Application: 7-segment display

Decimal digits display on 7-segment display:



Application: 7-segment display

To display decimal digits we have to design seven “segment decoder” circuits. That is, we will need a different circuit for each of the segments on the display.

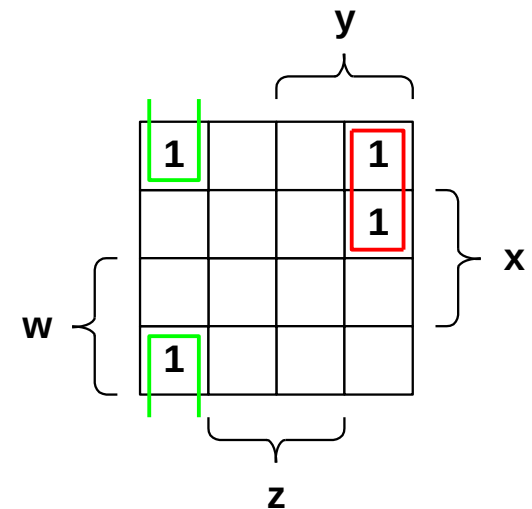
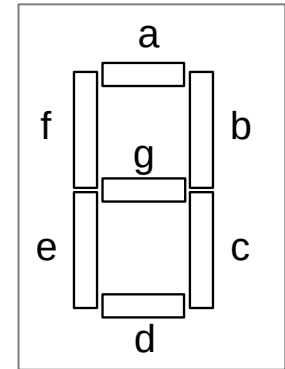


Using binary numbers (See Week 8!) we can code the numbers 0-9 using 4 digits, e.g. 5 = 0101, 8=1000.

Application: 7-segment display

Seven segment decoder truth table:

key	inputs				outputs						
	w	x	y	z	a	b	c	d	e	f	g
0	0	0	0	0					1		
1	0	0	0	1					0		
2	0	0	1	0					1		
3	0	0	1	1					0		
4	0	1	0	0					0		
5	0	1	0	1					0		
6	0	1	1	0					1		
7	0	1	1	1					0		
8	1	0	0	0					1		
9	1	0	0	1					0		

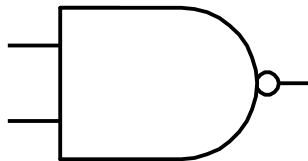


Seg e = $w'x'y'z' + w'x'yz' + w'xyz' + wx'y'z'$

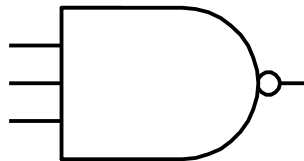
Min SOP for seg e = $x'y'z' + w'yz'$ – Karnaugh map

NAND gate

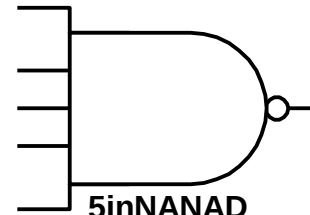
A NAND gate has two or more inputs. Its output is the negation of the product of the inputs. So it is the **negation of AND**.



2inNAND



3inNAND



5inNAND

x	y	$(x \cdot y)'$
0	0	1
0	1	1
1	0	1
1	1	0

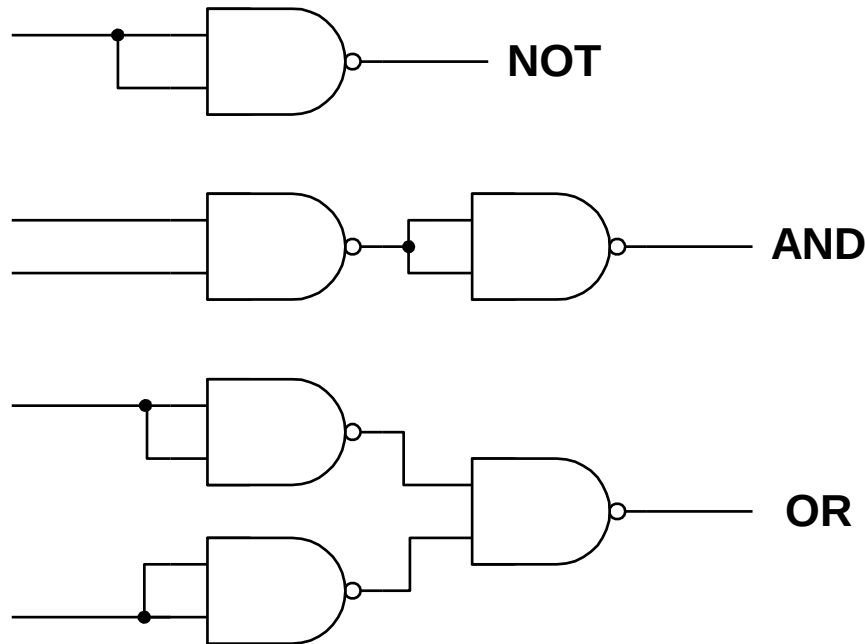
x	y	z	$(x \cdot y \cdot z)'$
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

One reason why a NAND gate is important is that it is a "universal gate", which means that **a logic circuit can be made entirely from NANDs**, as we shall see shortly.

The universality of NANDs

We can construct a circuit entirely from NANDs because we can construct AND, OR and NOT gates from NANDs, as shown below.

Therefore, in any AND-OR circuit, the AND, OR and NOT gates can be replaced by appropriate combinations of NANDs.



So a **NAND** is called a **"universal gate"**.

Converting an AND-OR circuit to NANDs only

One way to convert an AND-OR circuit to one that uses only NANDs is to replace each AND, OR and NOT by the equivalent NAND circuit, and then simplify where possible.

A simpler algebraic method uses de Morgan's laws.

We begin with the sum of products that represents the circuit.

We negate this expression twice and then use de Morgan's law on the inner negation to obtain an expression in terms of NANDs.

Converting an AND-OR circuit to NANDs only

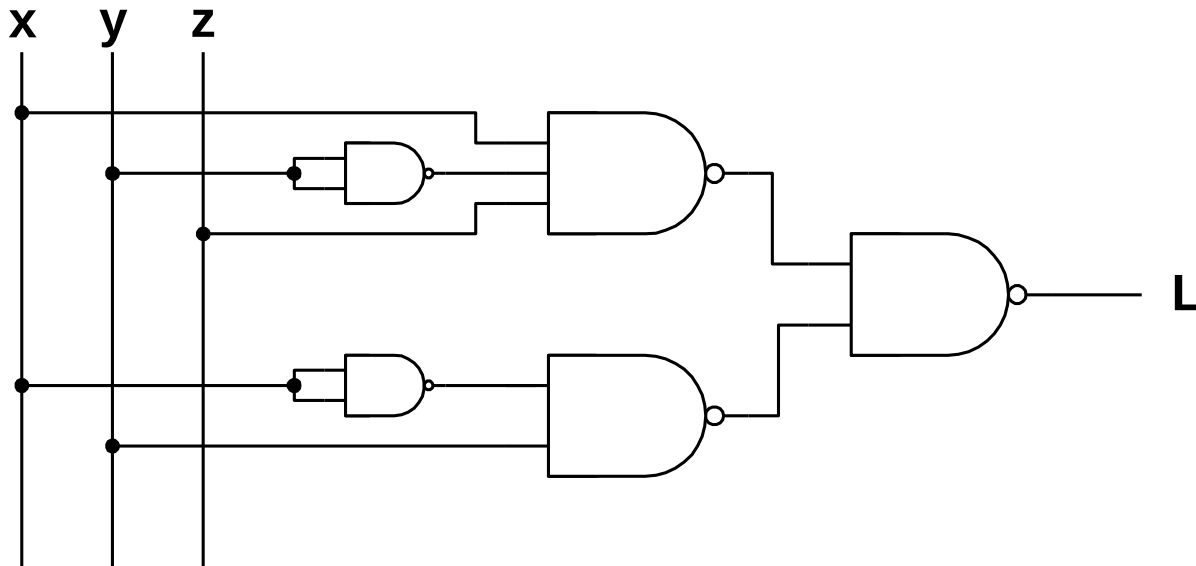
Example:

Suppose we want to change the sum of products

$$L = xy'z + x'y$$

into an expression in terms of NANDs. We negate the expression twice, and use de Morgan's law on the inner negation:

$$L = (xy'z + x'y) = [(xy'z + x'y)']' = [(xy'z)'(x'y)']'$$



The End